



ProjectIMS

System Documentation — public edition

System	ProjectIMS — QA and document control
Sector	Building-services engineering consultancy
Version	1.1
Status	Public edition
Date	3 September 2026
Platform	Django 6.0 · Python 3.12 · PostgreSQL
Scale	9 modules · 35 models · 69 routes · 342 tests

Client-neutral edition, published as a portfolio reference. Prepared from direct inspection of the implemented source code. All organisations, people and projects in examples are fictional. Contains no credentials or secrets.

ProjectIMS — System Documentation

A QA and document-control platform for building-services engineering.

ProjectIMS — QA and document control Internal Quality-Assurance and Document-Control Platform

Public edition. This is the client-neutral edition of the system documentation, published as a portfolio reference. The client's name, project data and commercial details have been removed. All organisations, people and projects appearing in examples are fictional.

The application is branded from configuration — see the `DSCIMS_BRAND_*` settings — so the product name used throughout is the demonstration brand, not a client's.

Document Control

FIELD	VALUE
Document title	ProjectIMS — System Documentation (public edition)
System name	ProjectIMS
Sector	Building-services engineering consultancy
Document version	1.1
Status	Public edition
Date of issue	3 September 2026
Prepared for	CTO, Technical Lead, Project Manager, incoming engineering team, QA and DevOps
Source of truth	The ProjectIMS source tree at the commit accompanying this document
Basis of preparation	Direct inspection of the implemented codebase — models, URLs, views, services, settings, templates and tests

Intended audience and how to read this document

READER	SUGGESTED ENTRY POINT
Client / Project sponsor	§1 Executive Overview, §3 Users & Permissions, §6 Workflows
CTO / Technical Lead	§2 Architecture, §7 Business Rules, §19 Known Limitations
Software engineer joining the project	§4 Modules, §8 Data Model, §17 Maintenance Guide
QA engineer	§5 Features, §6 Workflows, §15 Error Handling, §16 Scenarios
DevOps engineer	§13 Configuration, §14 Deployment, §18 Troubleshooting

A note on accuracy

Every statement in this document was checked against the implementation. Where behaviour is configuration-dependent, deliberately deferred, or not implemented, it is stated plainly rather than glossed over. Sections that record a **specification gap** identify a decision that was inferred from the available brief and still requires client confirmation.

No credentials, secret keys, tokens or production connection strings appear in this document. Configuration values are shown as placeholders.

Table of Contents

1. Executive Overview
 2. System Architecture
 3. Users, Roles & Permissions
 4. Module Documentation
 5. Complete Feature Documentation
 6. Complete User Workflows
 7. Business Logic & Rules
 8. Database & Data Model
 9. HTTP Interface (API) Documentation
 10. Administration & Management Features
 11. Notifications & Automated Processes
 12. Authentication & Security
 13. Configuration
 14. Deployment Architecture
 15. Error Handling & Edge Cases
 16. End-to-End Business Scenarios
 17. Maintenance & Future Development Guide
 18. Troubleshooting
 19. Known Limitations & Specification Gaps
 20. Glossary
 21. Appendix A — Complete Route Table
 22. Appendix B — Model Index
 23. Appendix C — Test Inventory
-

1. Executive Overview

1.1 Purpose

ProjectIMS is an internal, staff-only web application that governs how a building-services engineering consultancy runs quality assurance and document control across its project portfolio.

It exists to make three things reliable and auditable:

1. **That QA work was actually done** — every project runs through a templated checklist across seven engineering disciplines, and each phase is signed off by a manager using a PIN, with every attempt recorded.
2. **That compliance events are recorded and closed** — four registers (Milestones, Non-Conformances, Additional Scope, Customer Focus) track commitments, failures, out-of-contract work and client satisfaction.
3. **That the right drawings went to the right people at the right revision** — transmittals record what was issued, at which revision, to whom, and when, with recipient details frozen at the moment of issue.

1.2 The problem it solves

The practice previously ran on a legacy PHP (Yii 1.1) application bolted onto WordPress. The problems ProjectIMS is built to eliminate:

PROBLEM	HOW PROJECTIMS ADDRESSES IT
QA completion asserted rather than evidenced	Checklist state is per-activity and progress is computed, never stored
Sign-off with no audit trail	Every PIN attempt, successful or failed, is written to <code>SignoffAttempt</code>
The same QA phase tracked separately per discipline, drifting apart	Common phases are folded into every discipline and reconciled on sign-off
Drawings reissued at the same or an earlier revision	Revisions must strictly increase; enforced at form level and again at finalise
"Who did we send that to?" unanswerable after an address-book edit	Recipient name, email and delivery method are snapshotted at finalise
Notifications that may or may not have been sent	Every notification writes an audit row before dispatch is attempted
Plaintext password comparison in the legacy login	Django's password hashing; the sign-off PIN is hashed with the same hashers

1.3 Target users

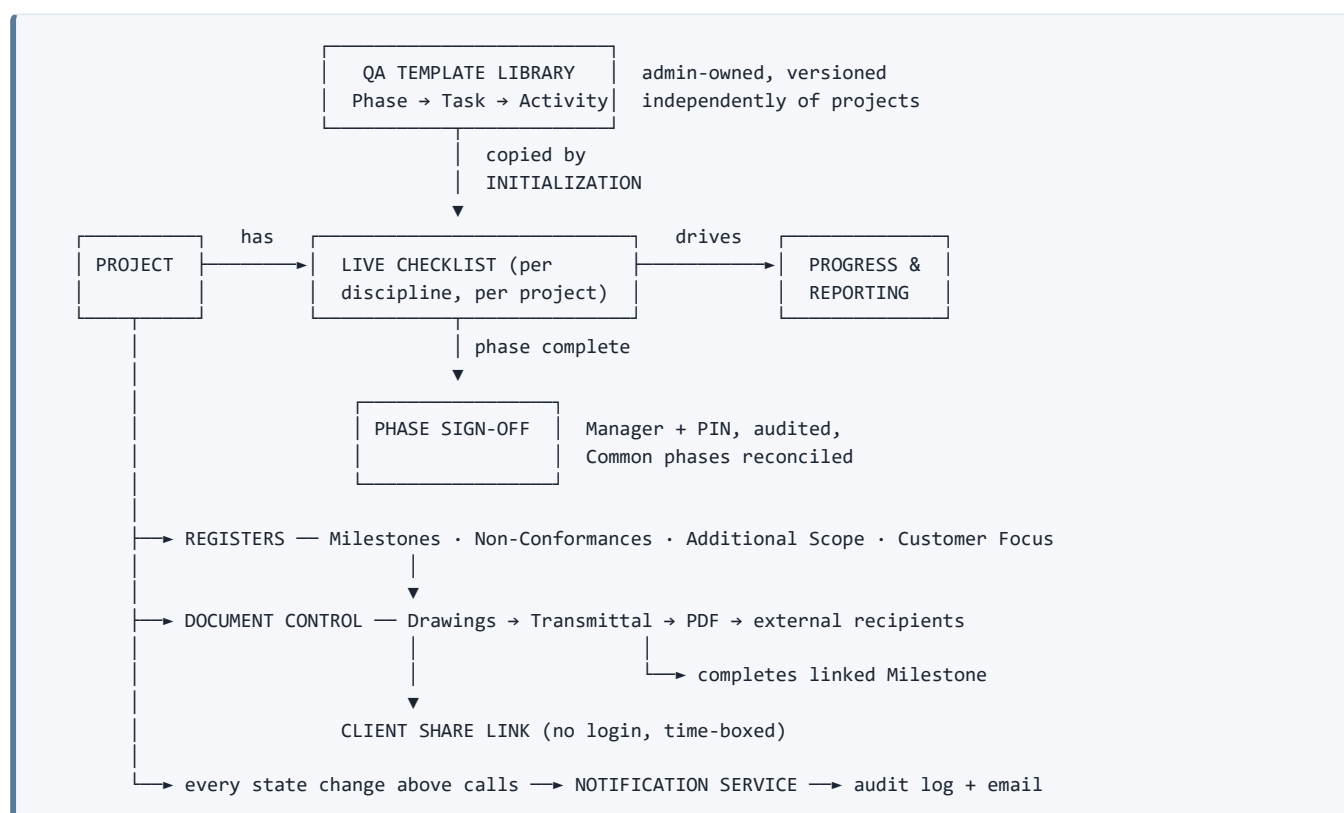
ProjectIMS is **staff-only**. Clients never log in. They receive documents by email and through a single time-boxed, unauthenticated share link.

USER GROUP	APPROXIMATE SIZE	PRIMARY USE
Engineers (Staff)	Majority of the practice	Working through discipline checklists, raising register entries
Managers	Directors, associates	Phase sign-off, portfolio reports
IMS Coordinators	One or two	QA template library, staff records, system settings, question bank
Clients & consultants	External	Receive transmittals by email; read-only share link

1.4 Main objectives

- Provide one templated QA process applied consistently to every project.
- Make QA progress visible at activity, phase, discipline, project and portfolio level.
- Enforce that nothing is marked finished without the date that finishes it.
- Guarantee that document issue is a matter of record, not recollection.
- Give management live portfolio reporting with print-ready output.

1.5 How the system works — high level



The single most important structural idea is the **template-versus-live split**. The QA template library is what the practice *intends* to do; a project's live checklist is a copy taken at initialization. Editing a template never rewrites work already recorded — a separate, explicit re-sync tool pushes corrections into live projects, and it carries wording and links only.

2. System Architecture

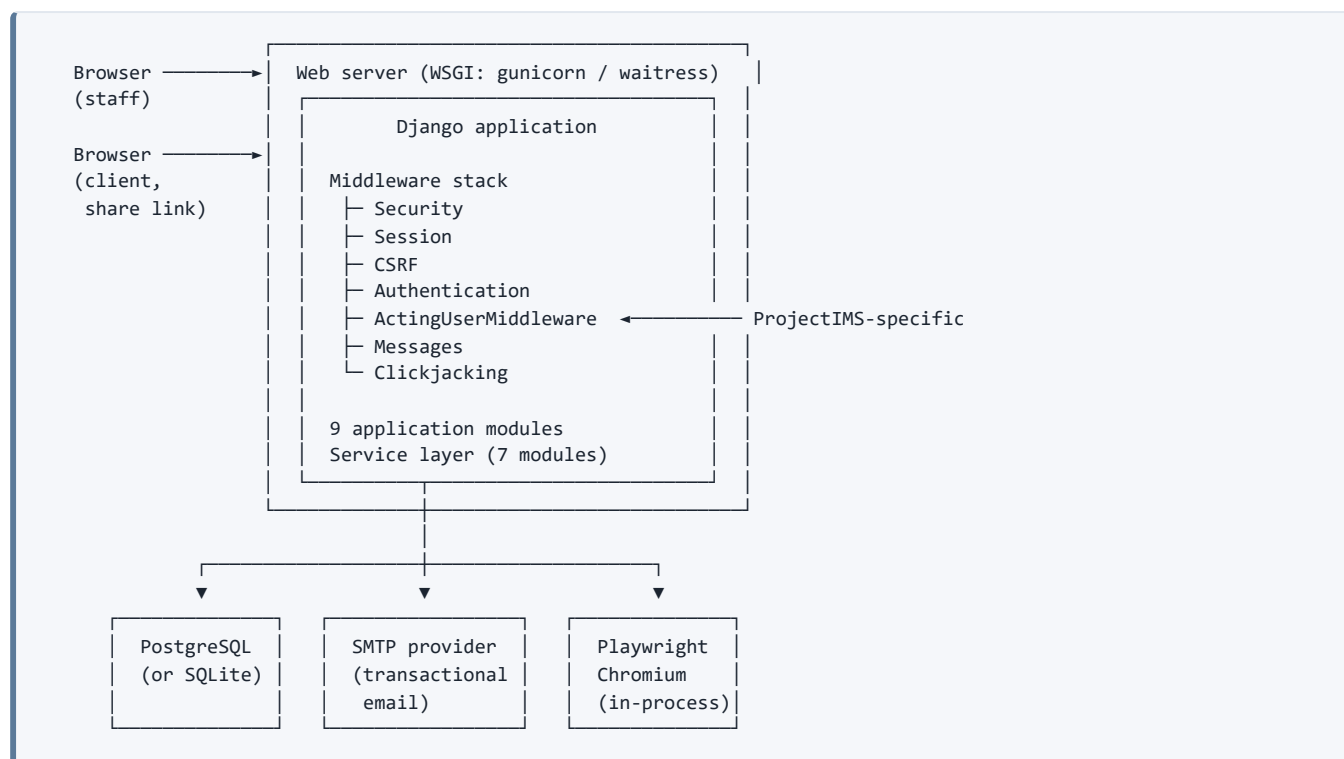
2.1 Technology stack as built

LAYER	TECHNOLOGY	NOTES
Language / runtime	Python 3.12.10	CPython
Web framework	Django 6.0.8	Pinned <code>Django>=6, <6.1</code>
Rendering	Django server-side templates	No SPA, no JavaScript build step
Interactivity	HTMX 2.0.4, Alpine.js 3.14.9	Loaded from CDN; no npm, no bundler
Styling	Hand-written CSS extracted from the approved design pack	No CSS framework
Database (development)	SQLite	<code>db.sqlite3</code> at repository root
Database (production)	PostgreSQL	Intended target; see §19
PDF generation	Playwright (Chromium)	Pluggable; see §2.6
Email	Django email backend	Console backend in development
Background processing	None	See §2.7
Object storage	None	See §2.8

Architectural note. The original build brief recommended a Next.js/Node/React stack. The client directed a Django implementation instead. The brief permitted substitution provided three architectural properties were preserved: a shared layout shell, an explicit template-versus-live data split, and a notification service every module calls into. All three are implemented and are described below.

2.2 Application topology

ProjectIMS is a **single deployable Django application**. There is no separate frontend service, no API gateway and no microservice decomposition.



2.3 Request lifecycle

1. The web server hands the request to Django's WSGI application.
2. `SessionMiddleware` restores the session from a cookie with a **one-year** lifetime (`SESSION_COOKIE_AGE = 31,536,000` seconds), refreshed on every request (`SESSION_SAVE_EVERY_REQUEST = True`). This implements the brief's "returning within a year signs you in automatically" requirement.
3. `CsrfViewMiddleware` validates the CSRF token on every unsafe method.
4. `AuthenticationMiddleware` populates `request.user` .
5. `ActingUserMiddleware` (ProjectIMS-specific) populates `request.acting_user` — see §2.4.
6. The view executes. Views enforce role permissions explicitly in Python.
7. Templates render server-side. HTMX endpoints return HTML fragments, not JSON.

2.4 Identity model: signed-in user versus acting user

This is a deliberate and load-bearing distinction that a maintainer must understand before changing any view.

CONCEPT	ATTRIBUTE	MEANING	USED FOR
Authenticated user	<code>request.user</code>	Who signed in with a password	Every permission decision
Acting user	<code>request.acting_user</code>	Who work is attributed to	Attribution only — <code>created_by</code> , <code>updated_by</code> , <code>recorded_by</code>

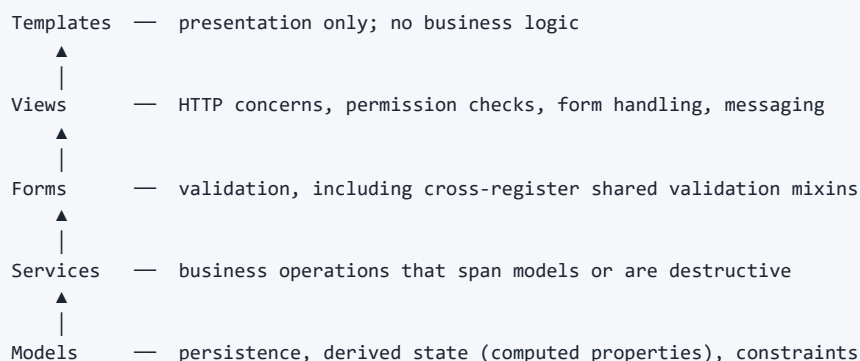
The topbar "Working with..." switcher changes the acting user without re-authentication, mirroring how the practice works in reality (one person at a shared workstation recording another's work). Implemented in `accounts/middleware.py` .

Security-critical rule: `acting_user` must never be consulted to decide whether something is permitted. Doing so would let any user grant themselves any permission by switching identity. Tests in

`accounts/tests/test_identity.py::SwitchingDoesNotGrantPermissionsTests` pin this.

Defensive behaviour: if the acting user is deactivated mid-session, the middleware silently falls back to the signed-in account and clears the session key, so work is never credited to a departed staff member.

2.5 Layer structure within the application



The **service layer** is where operations live that are transactional, destructive, or that trigger side effects:

SERVICE MODULE	RESPONSIBILITY
<code>qa/services.py</code>	Building a project's live checklist from templates; adding a discipline
<code>qa/signoff.py</code>	Phase sign-off, PIN verification, Common-phase reconciliation, revocation
<code>qa/resync.py</code>	Diffing and pushing template corrections into live checklists
<code>documents/services.py</code>	Transmittal locking and finalisation; milestone auto-completion
<code>notifications/services.py</code>	The single dispatch point for every notification
<code>reports/queries.py</code>	Portfolio aggregations, computed in bulk
<code>core/pdf.py</code>	Engine-agnostic document rendering

2.6 PDF generation subsystem

`core/pdf.py` exposes one function, `render_pdf(template_name, context)`, which returns a `RenderedDocument` carrying bytes, a content type, a file extension and the engine name. **No view names an engine.**

Engine selection is controlled by `DSCIMS_PDF_ENGINE`:

VALUE	BEHAVIOUR
<code>auto</code> (default)	Try WeasyPrint, then Playwright, then fall back to HTML
<code>weasyprint</code>	Force WeasyPrint
<code>playwright</code>	Force Playwright
<code>html</code>	Force the HTML fallback (used by the test runner)

Currently deployed on the development machine: Playwright. WeasyPrint was evaluated and cannot run on the Windows development host — it requires the GTK/Pango native libraries, which are not a pip package. Attempting to use it raises `OSError: cannot load library 'libgobject-2.0-0'`.

The HTML fallback is a deliberate design decision: a missing optional dependency degrades the output rather than breaking the workflow that produces it. The transmittal and report screens state on-screen which engine is live, so nobody is handed an HTML file believing it is a PDF.

2.7 Background processing — none implemented

There are no background workers, no task queue (no Celery, RQ or Django-Q), and no scheduled jobs (no cron, no `django-crontab`).

Every operation in ProjectIMS is synchronous within the HTTP request that triggers it, including email dispatch and PDF generation. The consequences are documented in §19 Known Limitations.

2.8 File and object storage — minimal

`MEDIA_ROOT` and `MEDIA_URL` are configured, but **no model currently stores an uploaded or generated file**. Specifically:

- Transmittal PDFs are generated **on demand** at each request and are not persisted.
- Drawings are recorded as metadata (number, title, discipline, revision history). The drawing files themselves are not uploaded to or served by ProjectIMS.
- No S3-compatible object storage is configured.

2.9 External services

SERVICE	STATUS	NOTES
SMTP / transactional email	Configured, provider not selected	Console backend in development
Google Fonts CDN	Active	IBM Plex Sans / IBM Plex Mono
cdnjs (HTMX, Alpine.js)	Active	Pinned versions
Chromium (Playwright)	Local binary	Launched in-process for PDF rendering

Operational note. Font and JavaScript assets load from public CDNs. An air-gapped or restricted-network deployment must vendor these locally; see §19.

2.10 Key dependencies

PACKAGE	VERSION	PURPOSE	CRITICALITY
Django	>=6, <6.1	Framework	Essential
playwright	>=1.60	PDF rendering	Pinned in requirements; HTML fallback exists
psycopg (to be added)	—	PostgreSQL driver	Required for production

The dependency surface is deliberately minimal. There is no `django-environ`, no `crispy-forms`, no REST framework and no CSS framework.

3. Users, Roles & Permissions

3.1 The single user model

All users are records of `accounts.Staff`, which extends Django's `AbstractUser`. There is no separate client or external user model — **clients have no account of any kind**.

ProjectIMS-specific fields on `Staff`:

FIELD	TYPE	PURPOSE
<code>category</code>	choice	Drives all role permissions
<code>position</code>	text	Job title, display only (e.g. "Associate Director")
<code>initials</code>	text	Avatar initials; derived from name when blank
<code>staff_number</code>	text, unique, read-only	Auto-allocated <code>ST-001</code> , <code>ST-002</code> , ...
<code>pin</code>	hashed	Sign-off PIN; never stored in plaintext
<code>is_active</code>	boolean	Soft-delete flag

3.2 The three roles

Roles are determined solely by `Staff.category`. Two computed properties express the permission model:

```
is_manager      = category in {manager, ims_coordinator}
is_administrator = category == ims_coordinator or is_superuser
```

An IMS Coordinator is therefore a superset of a Manager.

Terminology note. The business title is **IMS Coordinator** (as shown on the approved Staff Administration screen). The capability is named `is_administrator` throughout the code. They refer to the same thing.

3.3 Permission matrix

Legend: ● full access ○ read-only — no access

CAPABILITY	STAFF	MANAGER	IMS COORDINATOR
Navigation			
Dashboard, Projects, Companies	●	●	●
Reports section (sidebar)	—	●	●
Admin section (sidebar)	—	—	●
Projects			
View project list and overview	●	●	●
Create a project	●	●	●
Run the initialization wizard	●	●	●
Star/unstar a project (own favourites)	●	●	●
QA checklist			
View any discipline checklist	●	●	●
Change activity state	●	●	●
Change activity state on a signed-off phase	—	—	—
Open the phase sign-off screen	—	●	●
Confirm sign-off (requires signatory's PIN)	—	●	●
Remove ("un-sign") a sign-off	—	●	●
Registers			
View / create / edit Milestones	●	●	●
Raise / edit / resolve Non-Conformances	●	●	●
Raise / decide / close Additional Scope	●	●	●
Create and answer Customer Focus surveys	●	●	●
Document control			
View transmittals	●	●	●
Create, edit and finalise a transmittal	●	●	●
Manage the Document Register	●	●	●
Manage the project distribution list	●	●	●
Create / replace a client share link	●	●	●
Companies			
View, create and edit companies and contacts	●	●	●
Reports			
All five reports and PDF export	—	●	●

CAPABILITY	STAFF	MANAGER	IMS COORDINATOR
Administration			
QA Template Library (create/edit/delete)	—	—	•
Template re-sync to live projects	—	—	•
Customer Focus Question Master	—	—	•
Staff directory, edit, deactivate	—	—	•
System settings (email configuration)	—	—	•

Restrictions applying to every role

- **No hard delete exists in the interface for Staff, Projects, Companies or any register entry.** Staff are deactivated (soft delete). Template nodes are the only records with a delete control, and deleting one does not affect live checklists.
- **A user cannot deactivate their own account.** Enforced in `accounts.views.staff_deactivate`.
- **A signed-off phase is read-only** for all roles until the sign-off is removed.
- **A finalised transmittal is immutable** for all roles.
- **Switching acting identity grants no permissions.**

3.4 How permissions are enforced

Enforcement is explicit and in Python, at two levels:

1. **Navigation is hidden, not disabled** (per the brief's requirement) — the sidebar omits sections the role cannot use.
2. **Every gated view re-checks in code** and raises `PermissionDenied` (HTTP 403). Hiding a link is not access control.

There are **13 explicit role checks** across five view modules. All are covered by tests that assert a lower-privileged role receives HTTP 403, not merely a hidden link.

4. Module Documentation

ProjectIMS comprises nine Django applications. Apps live at the repository root; `dscims/` is the settings package only.

MODULE	PYTHON LOC	TEMPLATES	RESPONSIBILITY
<code>core</code>	658	5	Shared primitives, dashboard, settings, PDF, test runner
<code>accounts</code>	822	4	Staff model, authentication, identity switching, staff admin
<code>projects</code>	750	5	Project hub, taxonomy, services, favourites, initialization UI
<code>qa</code>	2,161	7	Template library, live checklist, sign-off, re-sync
<code>registers</code>	1,580	11	The four compliance registers and the question bank
<code>companies</code>	287	4	External address book
<code>documents</code>	1,715	9	Drawings, transmittals, document register, share links
<code>notifications</code>	396	1	Notification log, dispatch service, activity feed
<code>reports</code>	926	13	Five reports, shared filtering, portfolio aggregation
Total	10,354	62	35 models, 69 routes, 342 tests

4.1 `core`

Responsibility. Cross-cutting primitives that would otherwise be duplicated.

Key components

COMPONENT	PURPOSE
<code>models.Counter</code>	Named, gap-free integer sequences, incremented under a row lock
<code>models.EmailSettings</code>	Singleton runtime email configuration
<code>forms.RequiresCompletionDateMixin</code>	The "cannot complete without a date" rule, defined once
<code>pdf.py</code>	Engine-agnostic document rendering
<code>context_processors.acting_user</code>	Exposes acting identity and the switchable staff list to every template
<code>test_runner.DSCIMSTestRunner</code>	Pins the PDF engine to <code>html</code> during tests
<code>management/commands/seed_demo.py</code>	Development data only

Counter — why it exists. Register references (`MS-014` , `NC-231`) are globally sequential across the practice, not per project, and must never be reissued after a deletion. A row count cannot provide that. `Counter.next_value` uses `select_for_update()` inside a transaction.

Depends on: nothing within ProjectIMS. **Depended on by:** every module.

4.2 accounts

Responsibility. Who people are, how they sign in, and who work is credited to.

COMPONENT	PURPOSE
<code>models.Staff</code>	The only user model
<code>middleware.ActingUserMiddleware</code>	Resolves <code>request.acting_user</code>
<code>forms.StaffLoginForm</code>	Dropdown-based sign-in — no free-text usernames
<code>forms.PasswordChangeForm</code> / <code>PinChangeForm</code>	Both require the current password
<code>views.staff_*</code>	IMS Coordinator staff administration

Depends on: `core` (for `allocate_reference`). **Depended on by:** every module, via `AUTH_USER_MODEL`.

4.3 projects

Responsibility. The project record and its classification taxonomy — the hub every other module attaches to.

Contains `Discipline`, `QAType`, `ProjectType`, `EngagementType`, `Project`, `ProjectService` (discipline + assigned engineer) and `Favorite`.

Communicates with: `qa` (initialization wizard calls `qa.services`), `registers` and `documents` (register card counts on the overview).

4.4 qa

Responsibility. The QA engine — the largest and most intricate module.

It holds **two parallel model trees**:

TREE	MODELS	OWNER
Template	<code>GroupMaster</code> → <code>TaskMaster</code> → <code>SubTaskMaster</code>	IMS Coordinator
Live	<code>ProjectGroup</code> → <code>ProjectGroupTask</code> → <code>ProjectGroupSubTask</code>	Per project

Plus `PhaseSignoff` and `SignoffAttempt`.

Every live node keeps a nullable foreign key back to the template node it came from. That link is what makes the re-sync tool possible, and `SET_NULL` means deleting a template node leaves live checklists intact.

Depends on: `projects`, `notifications`.

4.5 registers

Responsibility. The four compliance registers and the Customer Focus question bank.

All four register models inherit an abstract `RegisterEntry` providing the project link, the auto-allocated reference and creator attribution — so the numbering and attribution behaviour is written once.

Depends on: `projects`, `core`, `notifications`. **Depended on by:** `documents` (transmittals link to milestones), `reports`.

4.6 companies

Responsibility. The external address book, deliberately separate from `Staff`. Keeping the two apart is what makes "clients never log in" a structural property rather than a policy.

`ProjectCompany` is the per-project distribution list — a subset of the global address book carrying the contact and delivery method that apply to *that job*.

4.7 documents

Responsibility. Everything client-facing.

COMPONENT	PURPOSE
<code>Drawing</code> , <code>DrawingRevision</code>	The practice's own drawings and their issue history
<code>TransmittalEvent</code> , <code>TransmittalItem</code> , <code>TransmittalRecipient</code>	An issue event
<code>BackgroundDocument</code>	Internal Document Register of consultants' drawings
<code>ShareLink</code>	Time-boxed unauthenticated client access
<code>services.py</code>	Locking, finalisation, milestone auto-completion

Depends on: `projects`, `companies`, `registers`, `core`, `notifications`.

4.8 notifications

Responsibility. The single dispatch point. **No other module calls** `send_mail`.

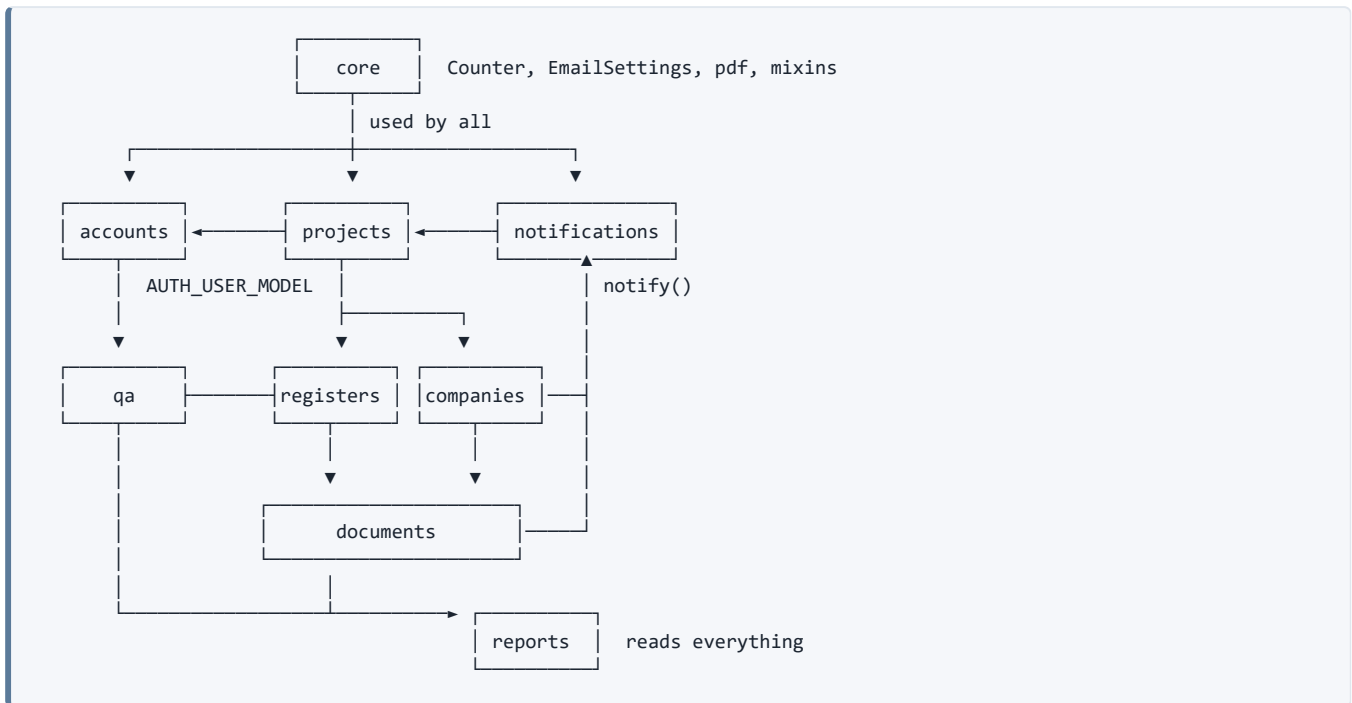
`notify()` writes the `NotificationLog` row **first**, then attempts dispatch. A send failure never propagates — the audit row is what must survive.

4.9 reports

Responsibility. Portfolio-level read-only reporting.

COMPONENT	PURPOSE
<code>forms.ReportFilterForm</code>	The one filter bar all five reports share
<code>queries.py</code>	Bulk aggregations
<code>templates/reports/print/</code>	Print-safe layouts with no interactive chrome

4.10 Module interaction map



5. Complete Feature Documentation

5.1 Authentication & Identity

5.1.1 Sign in

ATTRIBUTE	DETAIL
Purpose	Authenticate a staff member
Who	All staff
Entry point	<code>/login/</code>
Preconditions	An active <code>Staff</code> record with a usable password
Related	Identity switcher, session persistence

Workflow

1. User visits any URL; unauthenticated requests redirect to `/login/?next=...`.
2. The form presents a **dropdown of active staff**, ordered by surname.
3. The user selects their name and enters a password.
4. `StaffLoginForm.clean()` calls `authenticate()` with the selected staff member's username.
5. On success, Django establishes the session and redirects to `next` or the dashboard.

Business logic. There are no free-text usernames. The dropdown is a `ModelChoiceField`, so a value outside the active-staff queryset fails validation — a username cannot be guessed or enumerated by typing.

Validation rules

CONDITION	RESULT
No staff selected	Field error
Value not in the active-staff queryset	Field error
Password incorrect	"That password does not match the selected staff member."
Staff member deactivated	Not offered in the dropdown at all

Success: session established, one-year cookie issued. **Errors:** form redisplayed with the password cleared and the selection retained.

5.1.2 "Working with..." identity switcher

ATTRIBUTE	DETAIL
Purpose	Attribute work to a colleague without re-authenticating
Who	All authenticated staff
Entry point	Topbar dropdown (Alpine.js); posts to <code>/switch-identity/</code>
Preconditions	Authenticated

Workflow

1. The user opens the topbar pill and selects an active staff member.
2. `switch_identity` stores that id in `session["acting_user_id"]`.
3. A confirmation message states clearly that the user remains signed in as themselves.
4. Every subsequent write attributes to `request.acting_user`.

Business logic. Selecting yourself clears the session key entirely. **Permission behaviour:** none. Permissions continue to follow `request.user`.

5.1.3 Profile — password and PIN

Both forms require the **current account password** before any change. The PIN is stored using `make_password()` and verified with `check_password()`. An unset PIN never validates — including against an empty string.

5.2 Home Dashboard

ATTRIBUTE	DETAIL
Entry point	/
Who	All staff

Four statistic tiles (active projects, incomplete milestones, open non-conformances, open additional scope), a "My projects" favourites rail, and four register grids capped at five rows each. Every row navigates to its detail page.

The milestone panel is ordered **most urgent first** — overdue, then due soon. `Milestone.Meta` orders by `-due_date`, which is right for the register but backwards for a triage panel: it would surface milestones due in two months and bury the overdue ones. This is a "what needs attention" screen, not a report.

5.3 Project management

5.3.1 Project list and search

Searchable and filterable table: free text across number/title/client, plus status, discipline and project manager filters. Rows carry an HTMX favourite star that toggles without a page reload. `.distinct()` prevents a project matching two disciplines from appearing twice.

5.3.2 Project intake

Basic identification, classification, per-discipline checkbox + engineer pairing, and finance fields. On save the user is taken **directly into the initialization wizard** — implementing the brief's recommendation to consolidate the legacy "create then convert" two-step.

Disciplines are **optional at intake** (a project may be recorded before its scope is settled) but **required in the wizard**, where there would otherwise be nothing to build.

5.3.3 Project Overview

The hub screen: classification badges, four clickable register status cards with live counts, per-discipline progress bars linking into each checklist, the finance table, the client share link panel, document-control links and the engaged disciplines with assigned engineers.

5.3.4 Initialization wizard

ATTRIBUTE	DETAIL
Purpose	Build a project's live QA checklist from the template library
Entry point	/projects/<id>/initialize/
Preconditions	Template phases exist for the chosen disciplines
Destructive	Yes, when re-run

Workflow

1. **Step 1 — Disciplines.** At least one non-Common discipline required.
2. **Step 2 — Phases.** Offered phases are the union of the chosen disciplines' template phases and the Common discipline's, deduplicated by name and ordered by lifecycle position.
3. **Step 3 — Review & build.** Shows the selections and a warning whose severity depends on state: - Not yet initialized → **amber**: "building now is non-destructive". - Already initialized → **red**: every ticked activity, comment and sign-off on this project will be discarded.
4. On confirmation, `qa.services.initialize_project` runs in a transaction.

System processing (per selected discipline)

1. Delete all existing `ProjectGroup` rows for the project (cascades).
2. Ensure a `ProjectService` row exists for each chosen discipline.
3. For the discipline itself, then for the Common discipline: - Select template phases whose name is in the chosen set. - Skip a phase if this discipline already has one of that name — **a discipline's own phase wins over the folded-in Common one**, so the same phase never appears twice. - Create the `ProjectGroup`, inheriting `order` from the template. - Copy tasks, and copy only activities that **apply** to this project. - Skip a task entirely if every one of its activities was filtered out.

Wizard state is held in the session and cleared on completion.

5.4 QA Template Library

ATTRIBUTE	DETAIL
Who	IMS Coordinator only
Entry point	/admin/templates/

Two presentation modes:

- **All disciplines** — phases of the same name merged, each activity badged with the discipline that owns it (matching the approved artboard).
- **One discipline** — that discipline's tree alone, which is what adding and reordering require.

Full CRUD on phases, tasks and activities, including applicability (QA types / project types) and four reference links per activity.

Critical behaviour. Every screen states: *editing here shapes future initializations only*. Deleting a template node does **not** alter live checklists — the live node's `template` foreign key is `SET_NULL`, so the project keeps the activity it was built with and simply stops tracking a template that no longer exists.

5.5 Template re-sync

ATTRIBUTE	DETAIL
Purpose	Push template corrections into already-initialized projects
Who	IMS Coordinator
Entry point	<code>/admin/templates/resync/</code>

Select projects (optionally narrowed by discipline) → **Preview changes** shows a row-by-row diff (project, discipline, node type, field, from, to) → **Apply** writes them.

Scope, deliberately narrow. Re-sync carries **wording and reference links only**. It never adds or removes activities. Reshaping a checklist someone has already worked through — or signed off — is what re-initialization is for, and that warns first. Ticked progress survives a re-sync.

5.6 Live checklist

ATTRIBUTE	DETAIL
Entry point	<code>/projects/<id>/checklist/<discipline-code>/</code>
Who	All staff (edit); Managers additionally see sign-off

Discipline chips across the top, then a card per phase containing tasks and activity rows. Each activity offers five state controls: **Done**, **Started**, **N/A**, **Invalid**, **Clear**.

State changes post via HTMX; the row swaps itself and the phase header refreshes **out-of-band**, so progress and sign-off eligibility never lag behind the ticks.

Derived, never stored:

- **Task auto-close** — a task closes when every activity under it is Done or N/A. Shown read-only; there is no "close task" action that could drift.
- **Phase progress** — percentage of activities that are Done or N/A.

Common phases display a "Common" chip and name the other disciplines they affect.

5.7 Phase sign-off

ATTRIBUTE	DETAIL
Who	Manager or IMS Coordinator
Entry point	<code>/checklist/phase/<id>/signoff/</code>
Preconditions	Every task in the phase closed; signatory is a Manager with a PIN set

Workflow

1. A Manager opens the sign-off screen (a panel over the dimmed checklist).
2. Selects a **signing manager** — the dropdown offers only Manager-category staff **who have a PIN set**.
3. Enters the PIN and an optional comment.
4. If the phase is Common, an amber banner names every other discipline that will be reconciled.

System processing (`qa/signoff.py`)

1. `_rejection_reason()` validates — performing **no writes**.
2. On rejection: a failed `SignoffAttempt` is recorded and `SignoffError` raised.
3. On success, inside a transaction: a successful attempt, the `PhaseSignoff`, Common-phase reconciliation, and a notification.

Implementation note of consequence. Validation happens deliberately **outside** the transaction. The interface promises every attempt is logged; a failed attempt written inside an atomic block would be rolled back by the very exception that reports it, so only successes would ever survive. This was a real defect found and fixed during development, and it is pinned by tests.

Rejection reasons

CONDITION	MESSAGE
Already signed off	"This phase is already signed off."
A task remains open	"Every task in this phase must be closed before it can be signed off."
Signatory not a Manager	"Only Manager-category staff may sign off a phase."
Signatory has no PIN	" has no sign-off PIN set."
PIN incorrect	"That PIN is not correct."

Common-phase reconciliation. Signing a phase flagged `is_common` creates a mirrored `PhaseSignoff` on the same-named Common phase in every other discipline on that project, each linked by `reconciled_from`. A sibling that already carries a signature is **left alone** — someone else's signature is not ours to replace.

Revocation. Removing a sign-off lifts the whole reconciled set together, in either direction (from the original or from a mirror). Rows are retained with `revoked_at` and `revoked_by` set — removing a signature leaves as clear a trace as adding one.

Effect on the checklist. While a phase is signed off, its activities are **locked** for every role.

5.8 Compliance registers

All four share: a globally sequential reference, creator attribution, a **derived** status, and a completion date that gates closure.

5.8.1 Milestones

Dated issue commitments. Status derives from dates:

CONDITION	STATUS
<code>completion_date</code> set	Complete (green)
Not complete and <code>due_date</code> in the past	Overdue (red)
Not complete and due within 14 days	Due soon (amber)
Otherwise	Open (grey)

Two completion paths: manual (tick "mark complete" and supply a date) or **automatic** when every linked transmittal is finalised (§5.9.3).

Un-ticking "mark complete" clears the completion date, so a milestone is never open while carrying one — a state the status badge could not describe.

5.8.2 Non-Conformances

Raising an NC notifies **every staff member** — the broadest audience in the application — and the screen says so before you save.

Records four-party attribution (created by, identified by, **caused by**, resolved by) and four narrative fields (description, investigation, corrective action, preventive action), each with its own placeholder when empty.

Specification note. `caused_by` is **free text**, not a staff foreign key. The brief describes a "four-person attribution block", but the approved screen records causes such as *"Structural — column C4 relocation not communicated"*, which no person reference can express. See §19.

Resolution requires **both** a resolver and a resolved date. Editing an already-resolved NC prompts a confirmation before saving.

5.8.3 Additional Scope

A three-step lifecycle shown as a stepper: **Raised** → **Decision** → **Outcome**.

STATE	STATUS BADGE	CLOSED?
No decision	Awaiting decision (amber)	No
Pursue, no outcome	Pursuing (amber)	No
Not Pursue	Not pursued (grey)	Yes — no outcome needed
Pursue + Successful	Successful (green)	Yes
Pursue + Unsuccessful	Unsuccessful (red)	Yes

The outcome controls are visually disabled until a Pursue decision exists, **and the form rejects an outcome anyway** — so the rule holds even if the markup is bypassed.

5.8.4 Customer Focus

A per-project survey seeded from the firm-wide question bank, scored 0–10 per question, saved one answer at a time via HTMX.

Score = mean of *answered* questions × 10, expressed as a percentage. Unanswered questions are excluded rather than counted as zero, so a part-finished survey is not punished for being part-finished.

Auto-completion. The moment the score reaches the survey's target, the survey stamps `completed_at` and fires a notification. No manual action.

Snapshotting, twice over:

- The **target** is copied from the project at creation, so later changing the project's target cannot retroactively pass or fail a scored survey.
- The **question wording** is copied onto each answer, so editing or retiring a bank question never rewrites an answer already given.

Implementation note. `answered_scores` is a `cached_property`. `score`, `meets_target` and `points_from_target` render together; when each re-queried, a concurrent answer landing mid-render made the panel display a percentage and a distance-from-target that disagreed. A request that writes an answer drops the cache before checking the target.

5.9 Document control

5.9.1 Companies, contacts and distribution lists

A global address book (`Company` + `Contact`) and a per-project subset (`ProjectCompany`) carrying the contact and delivery method for that job — the same consultant may be emailed on one project and posted to on another.

Adding a company notifies all staff. Only one contact per company may be primary. Removing a company from a distribution list does **not** alter transmittals already issued.

5.9.2 Transmittals

ATTRIBUTE	DETAIL
Purpose	Issue drawings to external recipients as a matter of record
Entry point	<code>/projects/<id>/transmittals/</code>
Reference	TR<project-number>-<nnn> — per project , not global

Edit lock. Opening the editor takes a lock held for `DSCIMS_TRANSMITTAL_LOCK_MINUTES` (default 5) and refreshed on each visit. A second user sees a read-only view naming the holder and the minutes remaining. Locks **self-expire** — a lock requiring an administrator to clear would strand an issue whenever someone went home mid-edit.

Revision validation. Adding a drawing captures its current revision as `previous_revision`. The proposed revision must **strictly increase**. Ordering is `(length, uppercase value)`, which correctly gives `A < B < ... < Z < AA` and `1 < 2 < ... < 10` — plain string comparison gets both schemes wrong.

Finalising (`documents/services.py`) runs in a transaction:

1. Re-verify: at least one drawing, all revisions valid, at least one recipient.
2. **Snapshot the recipients** — company name, contact name, email and delivery method are copied onto the `TransmittalRecipient` rows.
3. Create a `DrawingRevision` for each item.
4. Stamp `finalized_at` / `finalized_by`; release the lock.
5. Render the PDF.
6. Fire two notifications: the **external Document Issue Notification** to the recipients and an **internal Finalised notice** to the project team.
7. Check the linked milestone for auto-completion.

A finalised transmittal is **immutable**.

5.9.3 Milestone auto-completion

When a transmittal is finalised, if it is linked to a milestone and **every** transmittal linked to that milestone is now finalised, the milestone's `completion_date` is set to today and a completion notification fires. The Milestones screen states this happens without a manual click.

5.9.4 Document Register (internal backgrounds)

Tracks consultants' drawings that the practice coordinates against — number, source discipline, description, revision, last updated.

This is the one notification not gated by the Settings on/off switch. The screen states it, and `notify(force=True)` implements it: coordinating against a superseded background is precisely the failure the register exists to prevent, so the notification is not something a setting can quietly suppress. Debug rerouting still applies, so forcing past the switch cannot start mailing real people from a developer's machine.

5.9.5 Client share link

The **only unauthenticated route** in the application.

PROPERTY	VALUE
Token	32 bytes from <code>secrets.token_urlsafe</code>
Lifetime	<code>DSCIMS_SHARE_LINK_DAYS</code> (default 90)
Revocation	Creating a new link revokes the previous one
Visible content	Finalised transmittals only — drafts never appear
Chrome	None — the template extends nothing, so it cannot inherit internal navigation

Expiry is enforced **on read**, not by a cleanup job that might not have run.

5.10 Reporting

Five reports sharing one filter bar (project-number range, year, QA type, PM, engineer, discipline, initialized-only, sort order):

REPORT	CONTENT
Non-Conformances	Every NC with attribution, dates and status
Additional Scope	Items split by lifecycle state
Customer Feedback	Scores against target, plus projects with no survey
Activity Status	Checklist completion by project and discipline
Summaries dashboard	Four tabs: Progress, Details, Performance, Team

`?export=pdf` renders the **same context** through a standalone print layout, so an exported report cannot quietly differ from the one on screen.

Overall project progress is computed across activities, not averaged across disciplines — a discipline with thirty activities outweighs one with three.

Team tab. Per-engineer assignments, project count and average progress. The assignment count and the average's denominator are **deliberately different numbers**: a discipline with no checklist yet is excluded from the average rather than counted as 0%, because nobody should read as behind on work that has not been set up.

5.11 Notification feed

`/notifications/` lists every notification grouped by day, showing the trigger, project, actor, audience and whether it was emailed, logged only, or rerouted. Sourced from the same rows the emails are built from, so the feed cannot

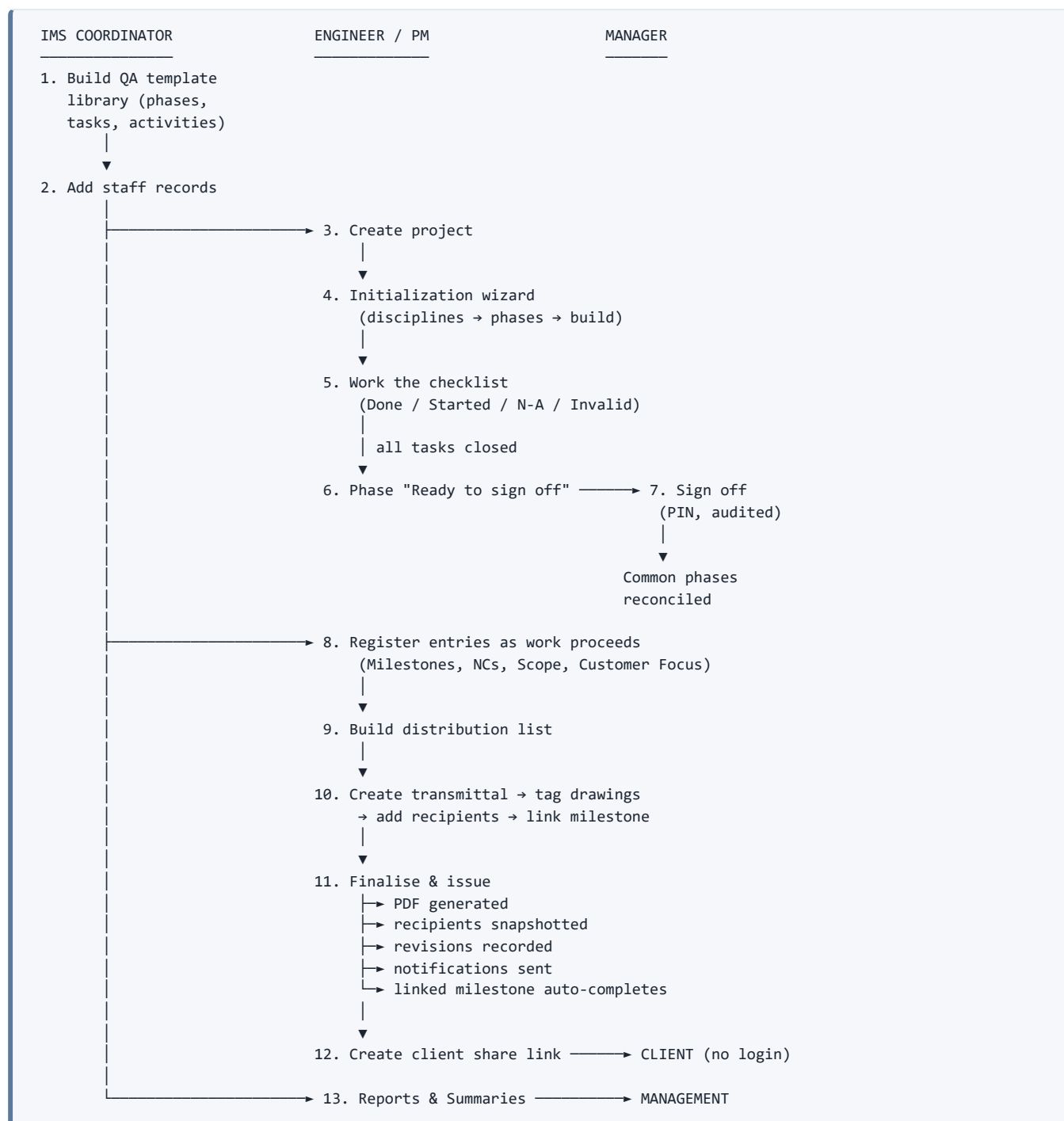
drift from what actually happened.

5.12 System settings

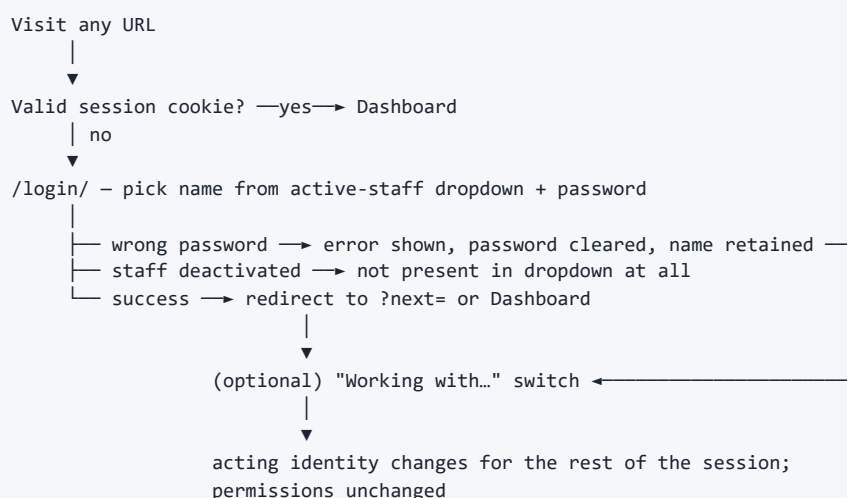
Runtime email configuration: master on/off switch, From name, From address, and a debug "reroute all mail" field. Loud banners appear when mail is rerouted or when notifications are switched off.

6. Complete User Workflows

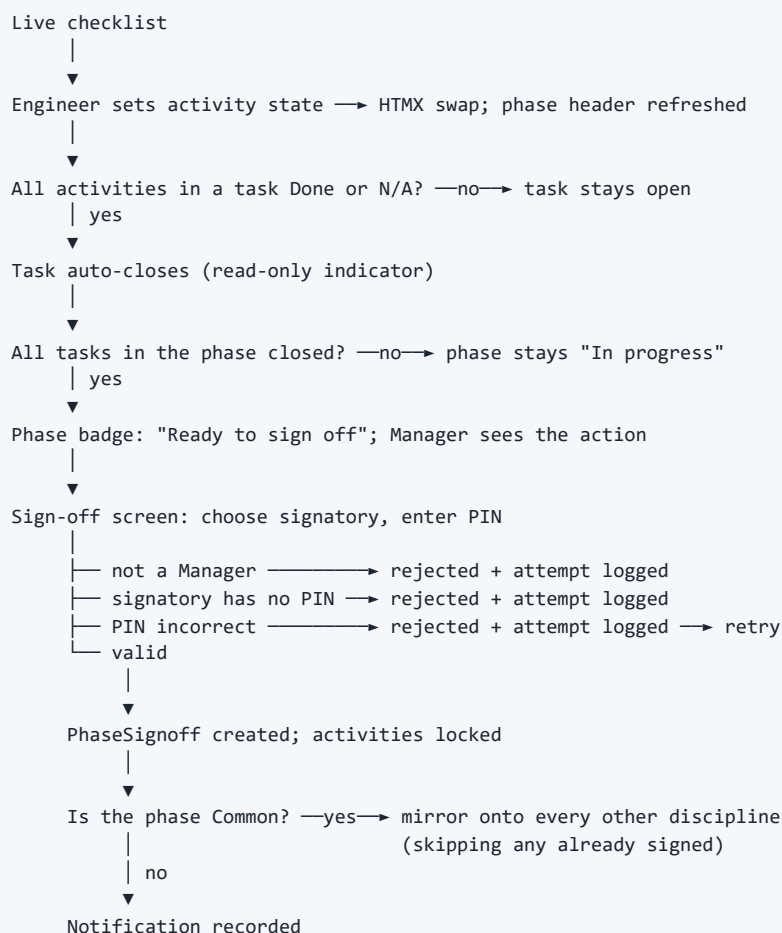
6.1 Primary flow — from empty system to issued document



6.2 Sign-in flow, including alternatives



6.3 Checklist to sign-off, including rejection paths



6.4 Non-Conformance lifecycle, including the closed-record path

```

Raise NC (summary, description, identified by, caused by)
  |
  ▼
Reference NC-nnn allocated; ALL STAFF notified
  |
  ▼
Owner records investigation / corrective / preventive action
  |
  ▼
Mark resolved — resolver missing? → "Record who resolved this..."
                └─ date missing? → "A resolved date is required..."
  |
  ▼ both supplied
Resolved; ALL STAFF notified again
  |
  ▼
Need to edit a closed NC? → confirmation prompt before saving

```

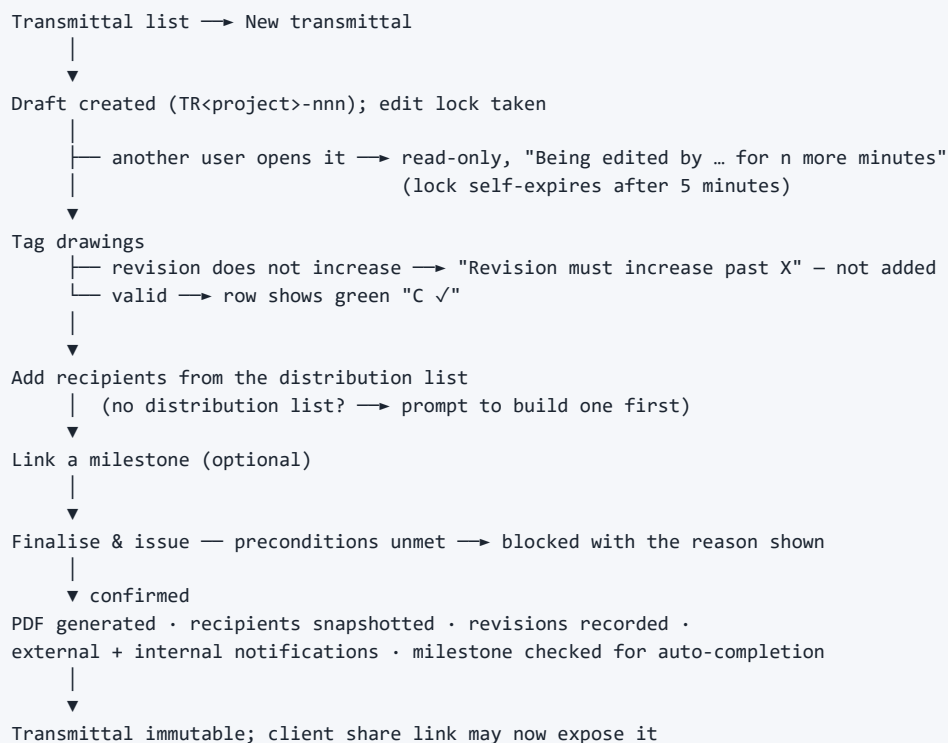
6.5 Additional Scope, including the cancellation path

```

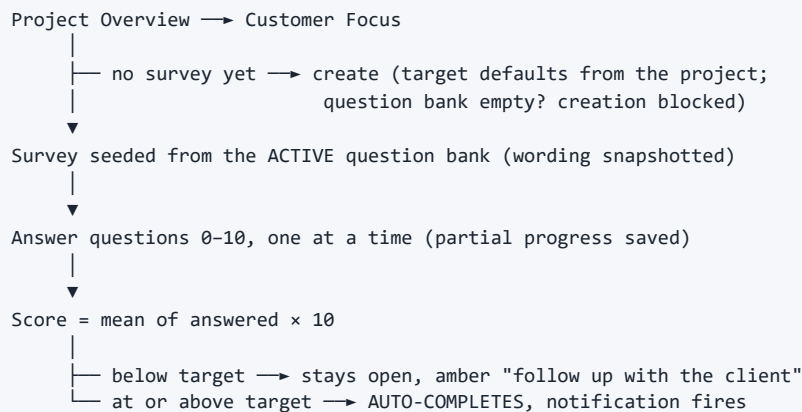
Raise AS item (summary, scope description) → project team notified
  |
  ▼
project manager decision
  ├── Not Pursue → CLOSED. No outcome required or possible.
  └── Pursue
      |
      ▼
      Work proceeds
      |
      ▼
      Record outcome — completion date missing? → rejected
      |
      ├── Successful → closed (green)
      └── Unsuccessful → closed (red)

```

6.6 Transmittal issue, including the lock and revision rejection paths



6.7 Customer Focus survey



7. Business Logic & Rules

This section catalogues the rules a maintainer must not break.

7.1 Identity and permissions

#	RULE	WHERE ENFORCED
BR-01	Permissions follow <code>request.user</code> , never <code>request.acting_user</code>	All views; pinned by tests
BR-02	A deactivated acting user falls back to the signed-in account	<code>accounts/middleware.py</code>
BR-03	Only Manager-category staff may sign off or un-sign a phase	<code>qa/views.py</code> , <code>qa/signoff.py</code>
BR-04	Template library, question bank, staff admin and settings are IMS Coordinator only	13 explicit checks
BR-05	A user cannot deactivate their own account	<code>accounts/views.py</code>
BR-06	Reports require Manager or above	<code>reports/views.py</code>

7.2 Referencing and numbering

#	RULE	DETAIL
BR-07	<code>MS/NC/AS/CF</code> references are global across the practice	<code>core.Counter</code>
BR-08	<code>ST-nnn</code> staff numbers are global and stable across saves	<code>Staff.save()</code>
BR-09	<code>TR<project>-nnn</code> transmittal references are per project	Counter key includes project number
BR-10	A reference is never reissued after deletion	Counter is not a row count

7.3 QA checklist

#	RULE	DETAIL
BR-11	A task closes when every activity is Done or N/A	Computed, read-only
BR-12	Invalid does not close a task	Superseded work is not finished work
BR-13	Progress = settled ÷ total activities	Invalid counts in the denominator
BR-14	Done and Invalid are mutually exclusive by construction	A single <code>state</code> field, not two flags
BR-15	A discipline's own phase wins over the folded-in Common one	Initialization
BR-16	Phase order is a global lifecycle position	Otherwise Common ties with each discipline at 0
BR-17	An activity applies if its type lists are empty or contain the project's	<code>SubTaskMaster.applies_to()</code>
BR-18	A task whose activities all filter out is not created	Initialization
BR-19	Re-initialization discards all progress	Warned in red at step 3
BR-20	Re-sync carries wording and links only	Never adds or removes activities
BR-21	A signed-off phase is read-only	<code>qa/views.py</code>

7.4 Sign-off

#	RULE	DETAIL
BR-22	Every attempt is recorded, successful or not	Validation runs outside the transaction
BR-23	A phase with no tasks cannot be signed	Nothing to attest to
BR-24	Signing a Common phase reconciles across disciplines	<code>reconciled_from</code> links the set
BR-25	An existing signature is never overwritten	Reachable when a discipline is added later
BR-26	Revoking lifts the whole reconciled set, in either direction	Otherwise the record would lie
BR-27	Revoked sign-offs are retained, not deleted	<code>revoked_at</code> , <code>revoked_by</code>

7.5 Registers

#	RULE	DETAIL
BR-28	Nothing is "complete" without its completion date	<code>RequiresCompletionDateMixin</code> , shared
BR-29	Status is derived from dates and decisions, never stored	All four registers
BR-30	A resolver without a date does not close an NC	The date closes the record, not the intent
BR-31	Un-ticking "mark complete" clears the completion date	No "open with a completion date" state
BR-32	An outcome requires a Pursue decision	Enforced in the form, not just the UI
BR-33	"Not Pursue" closes an item with no outcome	
BR-34	CF score excludes unanswered questions	Not counted as zero
BR-35	CF target is snapshotted at creation	Project changes cannot re-score history
BR-36	CF question wording is snapshotted per answer	Bank edits never rewrite answers
BR-37	A survey auto-completes on reaching target	No manual action

7.6 Document control

#	RULE	DETAIL
BR-38	A drawing revision must strictly increase	Form + finalise; ordering is (length, value)
BR-39	A revision may never be issued twice for a drawing	Database unique constraint
BR-40	Finalising requires ≥ 1 drawing, all revisions valid, ≥ 1 recipient	<code>can_finalize()</code>
BR-41	Recipients are snapshotted at finalise	Address-book edits cannot rewrite history
BR-42	A finalised transmittal is immutable	
BR-43	Edit locks self-expire after 5 minutes	Never require manual clearing
BR-44	Only the lock holder may release it	
BR-45	A milestone auto-completes when all linked transmittals are finalised	
BR-46	Background changes always notify, bypassing the master switch	<code>notify(force=True)</code>
BR-47	Share links expose finalised transmittals only	Drafts never leave the practice
BR-48	Creating a share link revokes the previous one	A forwarded link cannot outlive its replacement
BR-49	Share-link expiry is checked on read	

7.7 Notifications

#	RULE	DETAIL
BR-50	The audit row is written before dispatch	
BR-51	A send failure never propagates	It must not roll back the record that triggered it
BR-52	Intended recipients are recorded even when sending is off	The log shows who <i>would</i> have been told
BR-53	Debug rerouting applies even to forced notifications	

7.8 Reporting

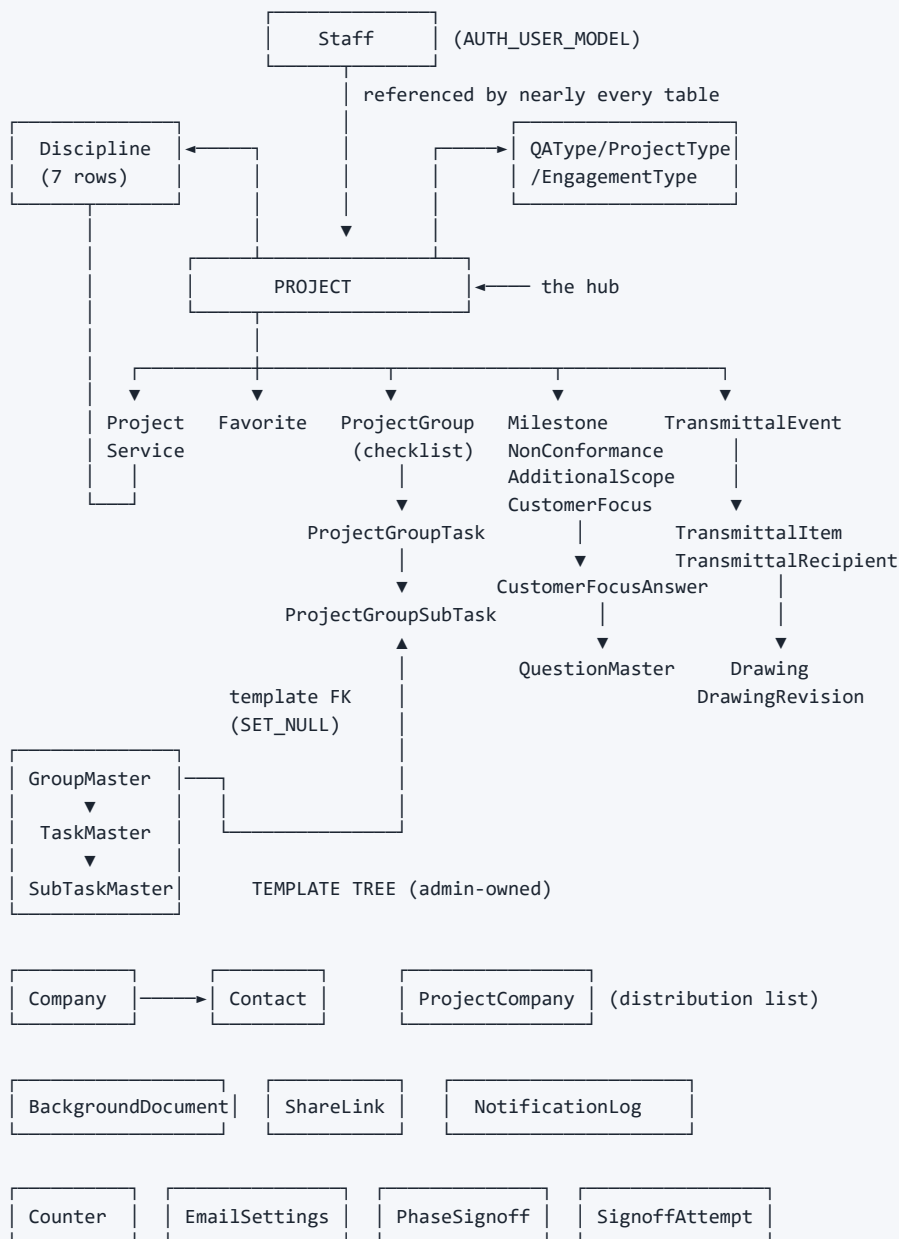
#	RULE	DETAIL
BR-54	An invalid filter returns everything, not an error	Read-only screens: too much beats none
BR-55	Overall progress is computed across activities	Not averaged across disciplines
BR-56	Team average excludes disciplines with no checklist	Not counted as 0%
BR-57	Export and screen share one context	They cannot disagree

8. Database & Data Model

8.1 Overview

35 models across 9 applications, 16 migration files. The schema is fully normalised with explicit `on_delete` behaviour on every foreign key.

8.2 Entity relationship overview



8.3 Relationship cardinalities

RELATIONSHIP	TYPE	ON DELETE
Project → ProjectService	1:N	CASCADE
Project → ProjectGroup	1:N	CASCADE
ProjectGroup → ProjectGroupTask → ProjectGroupSubTask	1:N:N	CASCADE
GroupMaster → TaskMaster → SubTaskMaster	1:N:N	CASCADE
ProjectGroupSubTask → SubTaskMaster (<code>template</code>)	N:1	SET_NULL
SubTaskMaster ↔ QAType / ProjectType	M:N	—
Milestone ↔ Discipline	M:N	—
Staff ↔ Project (via Favorite)	M:N	CASCADE
Project ↔ Company (via ProjectCompany)	M:N	CASCADE
ProjectGroup → PhaseSignoff	1:N	CASCADE
PhaseSignoff → PhaseSignoff (<code>reconciled_from</code>)	self-referential	SET_NULL
Drawing → DrawingRevision	1:N	CASCADE
TransmittalEvent → Item / Recipient	1:N	CASCADE
TransmittalEvent → Milestone	N:1	SET_NULL
CustomerFocus → CustomerFocusAnswer	1:N	CASCADE
CustomerFocusAnswer → QuestionMaster	N:1	SET_NULL

The `on_delete` policy, and why it matters

POLICY	USED FOR	RATIONALE
CASCADE	Children of a project or a parent record	The child has no meaning without the parent
PROTECT	Taxonomy (<code>Discipline</code> , <code>QAType</code>), <code>dsc_pm</code> , sign-off <code>signatory</code> , <code>transmittal drawing</code>	Prevents silently destroying referenced history
SET_NULL	Attribution (<code>created_by</code> , <code>updated_by</code>), template links, snapshotted recipients	The record survives; only the link is lost

`SET_NULL` on `TransmittalRecipient.company` is deliberate and important: if a company is deleted, the transmittal still records the snapshotted `company_name` , `contact_name` and `contact_email` .

8.4 Database constraints

CONSTRAINT	TABLE	GUARANTEES
<code>unique_project_discipline</code>	ProjectService	One service row per discipline per project
<code>unique_staff_favorite</code>	Favorite	No duplicate stars
<code>unique_project_discipline_phase</code>	ProjectGroup	A phase name appears once per discipline per project
<code>unique_project_company</code>	ProjectCompany	A company appears once on a distribution list
<code>unique_project_drawing</code>	Drawing	Drawing numbers unique within a project
<code>unique_drawing_revision</code>	DrawingRevision	A revision is never issued twice
<code>unique_event_drawing</code>	TransmittalItem	A drawing appears once per transmittal
<code>unique_project_background</code>	BackgroundDocument	Background numbers unique within a project

Important limitation. The database enforces that a revision is never *repeated*. The rule that a revision must strictly *increase* cannot be expressed as a check constraint, because it compares a row against its predecessors. It is enforced in `TransmittalItemForm.clean()` and again in `finalize_transmittal()`. This split is intentional and documented.

8.5 Data lifecycle

ENTITY	CREATED	MODIFIED	ENDS
Staff	By IMS Coordinator	Details, category, PIN	Deactivated, never deleted
Project	Intake form	Throughout its life	Status Complete/Archived; never deleted in the UI
ProjectGroup tree	Initialization	Activity states, comments	Deleted and rebuilt on re-initialization
PhaseSignoff	Sign-off	<code>revoked_at</code> on un-sign	Retained permanently
SignoffAttempt	Every attempt	Never	Retained permanently
Register entries	User action	Until closed	Closed by date; never deleted in the UI
TransmittalEvent	Draft creation	Until finalised	Immutable after finalising
DrawingRevision	Transmittal finalise	Never	Permanent record
ShareLink	User action	Revoked on replacement	Expires by date
NotificationLog	Every notification	<code>was_emailed</code> only	Retained permanently

Retention note. No purge, archival or retention policy is implemented. Audit tables grow without bound. See §19.

9. HTTP Interface (API) Documentation

9.1 There is no REST or GraphQL API

ProjectIMS is a **server-rendered application**. It exposes **no JSON API**, no token authentication, no API versioning and no OpenAPI schema. There is no Django REST Framework dependency.

All 69 routes return HTML and are authenticated by **session cookie**. This section documents the HTTP contract as implemented, because an integrator or QA engineer needs it — but it should not be mistaken for a public API.

9.2 Global conventions

ASPECT	BEHAVIOUR
Authentication	Session cookie; unauthenticated requests to protected routes → 302 to <code>/login/?next=...</code>
Authorisation	Explicit role checks → 403 (<code>PermissionDenied</code>)
CSRF	Required on every POST; missing/invalid → 403
Content type	<code>text/html</code> (or <code>application/pdf</code> on export routes)
Method enforcement	19 state-changing views are <code>@require_POST</code> ; a GET returns 405
Not found	404 for unknown records and invalid share tokens

9.3 HTMX partial endpoints

These return **HTML fragments**, not JSON. They are the closest thing to an internal API.

METHOD	ENDPOINT	PURPOSE	RETURNS
POST	<code>/projects/<pk>/favorite/</code>	Toggle a favourite	The swapped star control
POST	<code>/checklist/activity/<pk>/state/</code>	Set an activity state	The activity row, plus the phase header out-of-band
POST	<code>/registers/.../customer-focus/answer/<pk>/</code>	Record a score	The question row, plus the score panel out-of-band

Example — set activity state

```
POST /checklist/activity/57/state/
Cookie: sessionid=<session>
X-CSRFToken: <token>
HX-Request: true
Content-Type: application/x-www-form-urlencoded

state=done
```

RESPONSE	MEANING
200 + fragment	State applied; row and phase header returned
200 + row with "Locked" badge	Phase is signed off; no change made
405	Method was not POST
302	Not authenticated

An unrecognised `state` value is **silently ignored** — the activity is left unchanged rather than erroring.

9.4 Document endpoints

METHOD	ENDPOINT	AUTH	RETURNS
GET	<code>/projects/<pk>/transmittals/<pk>/pdf/</code>	Session	<code>application/pdf</code> (or <code>text/html</code> on the fallback)
GET	<code>/reports/<report>/?export=pdf</code>	Session + Manager	<code>application/pdf</code> (or <code>text/html</code>)
GET	<code>/share/<token>/</code>	None	<code>text/html</code> ; 404 if expired, revoked or unknown

`Content-Disposition` is `inline; filename="<name>.<ext>"`, with the extension following the engine actually used. Filenames are plain ASCII, because a non-ASCII filename forces RFC 2047 encoding of the header, which some clients render literally.

9.5 Full route inventory

See **Appendix A** for the complete table of all 69 routes.

10. Administration & Management Features

All administration is restricted to **IMS Coordinators**.

AREA	ROUTE	CAPABILITIES
QA Template Library	<code>/admin/templates/</code>	Create, edit, delete phases/tasks/activities; set applicability; attach four reference links
Template re-sync	<code>/admin/templates/resync/</code>	Select projects, preview a diff, apply wording and link corrections
Question Master	<code>/admin/questions/</code>	Add questions; retire/reactivate them
Staff directory	<code>/admin/staff/</code>	View all staff including deactivated
Staff detail	<code>/admin/staff/<id>/</code>	Edit name, initials, email, category, position
Staff deactivation	POST	Soft delete and reactivate; cannot target own account
System settings	<code>/admin/settings/</code>	Email master switch, From name/address, debug reroute

10.1 Django admin

Django's built-in admin is mounted at `/django-admin/` and is available to superusers. **No `ModelAdmin` classes are registered**, so it exposes only Users/Groups. It is not the intended administration surface and could be removed in production if desired.

10.2 Management commands

COMMAND	PURPOSE	SAFE IN PRODUCTION?
<code>seed_demo</code>	Creates demo templates, projects, companies, drawings and questions	No — development only

Standard Django commands (`migrate` , `collectstatic` , `createsuperuser` , `test`) behave normally.

11. Notifications & Automated Processes

11.1 Architecture

Every notification passes through `notifications.services.notify()` . It:

1. Resolves the audience from the `AUDIENCES` table.
2. Writes the `NotificationLog` row — **always, first**.
3. Dispatches email, unless suppressed.
4. Marks `was_emailed` on success.

A dispatch failure is swallowed: the audit row is what must survive, and a mail server outage must not roll back the non-conformance someone just raised.

11.2 The notification matrix as implemented

TRIGGER	FIRED WHEN	AUDIENCE	STRATEGY
<code>nc_raised</code>	A non-conformance is created	All staff	<code>all_staff</code>
<code>nc_resolved</code>	A non-conformance is resolved	All staff	<code>all_staff</code>
<code>as_raised</code>	Additional scope is raised	Project team	<code>project</code>
<code>as_decided</code>	A Pursue/Not Pursue decision is recorded	Project team	<code>project</code>
<code>as_outcome</code>	An outcome is recorded	Project team	<code>project</code>
<code>milestone_saved</code>	A milestone is saved with notify-on-save ticked	Project team	<code>project</code>
<code>milestone_completed</code>	A milestone completes (manually or automatically)	Project team	<code>project</code>
<code>transmittal_finalised</code>	A transmittal is finalised (internal notice)	Project team	<code>project</code>
<code>document_issue</code>	A transmittal is finalised (external notice)	Transmittal recipients	<code>explicit</code>
<code>company_created</code>	A company is added	All staff	<code>all_staff</code>
<code>background_changed</code>	A background is added or edited	All staff	<code>all_staff</code> (forced)
<code>phase_signed_off</code>	A phase is signed off	Recorded only	<code>none</code>
<code>phase_unsigned</code>	A sign-off is removed	Recorded only	<code>none</code>
<code>cf_completed</code>	A survey reaches its target	Project team	<code>project</code>

Audience strategies

STRATEGY	RECIPIENTS
<code>all_staff</code>	Every active staff member with an email address
<code>project</code>	The project's project manager plus every assigned engineer
<code>explicit</code>	Addresses passed by the caller (transmittal recipients)
<code>none</code>	Nobody — recorded in the feed only

Specification gap. The build brief refers to a PRD "notification matrix" that was not supplied. The audiences above are a first pass derived from the brief's per-module rules, and each entry in `AUDIENCES` names the rule it came from or states that it is an inference. Reconciling against the PRD is a matter of editing one dictionary. See §19.

11.3 Suppression rules

CONDITION	EFFECT
<code>EmailSettings.is_enabled</code> is off	Logged, not sent — except forced triggers
<code>EmailSettings.reroute_to</code> is set	Sent to that address only; real recipients still recorded
No recipients resolved	Logged, not sent
SMTP failure	Logged with <code>was_emailed = False</code>

Exactly one trigger is forced: `background_changed` .

11.4 Automated processes (event-driven, not scheduled)

PROCESS	TRIGGER	EFFECT
Common-phase reconciliation	Phase sign-off	Mirrors the signature across disciplines
Task auto-close	Activity state change	Derived — no write
Milestone auto-completion	Transmittal finalise	Sets <code>completion_date</code> , notifies
Customer Focus auto-completion	Answer saved	Sets <code>completed_at</code> , notifies
Share-link revocation	New link created	Revokes the previous link
Edit-lock expiry	Passage of time	Evaluated on read — no job required

All are synchronous, inside the triggering request. There are no scheduled jobs and no background workers.

12. Authentication & Security

12.1 Authentication

ASPECT	IMPLEMENTATION
Mechanism	Django session authentication
User model	<code>accounts.Staff (AbstractUser)</code>
Password storage	Django's configured hashers (PBKDF2 by default)
Password validators	Similarity, minimum length, common-password, numeric
Username entry	Not possible — selection from a dropdown of active staff
Session lifetime	One year, refreshed on every request
Logout	Flushes the session

12.2 Sign-off PIN

A second factor for the specific act of signing off a phase.

- Hashed with `make_password()` ; verified with `check_password()` .
- **Never stored or logged in plaintext.**
- An unset PIN never validates, including against an empty string.
- Setting or changing a PIN requires the current account password.
- Every attempt — successful or failed, with its reason — is written to `SignoffAttempt` .

Limitation. There is **no rate limiting or lockout** on PIN attempts. Attempts are recorded but not throttled. See §19.

12.3 Authorisation

Two layers, as required by the brief:

1. **Interface** — the sidebar hides sections a role cannot use.
2. **Server** — every gated view re-checks and raises `PermissionDenied` .

Object-level scoping is applied where records are nested under a project: for example

`get_object_or_404(NonConformance, pk=nc_pk, project=project)` prevents reaching another project's record by changing an id in the URL.

12.4 Session and CSRF

`CsrfViewMiddleware` is active; every form and every HTMX POST carries a token. No endpoint is CSRF-exempt.

Production hardening required. `SESSION_COOKIE_SECURE` , `CSRF_COOKIE_SECURE` , `SECURE_SSL_REDIRECT` and HSTS are **not currently set**. See §14.

12.5 Input validation

Validation is layered: Django form fields, model field constraints, custom `clean()` methods (revision increase, outcome gating, contact/company matching), shared mixins (`RequiresCompletionDateMixin`), and database constraints as the final backstop.

Templates auto-escape by default; there is no use of `|safe` on user-supplied content.

12.6 The unauthenticated share route

The single largest external surface, and deliberately narrow:

CONTROL	IMPLEMENTATION
Token entropy	32 bytes via <code>secrets.token_urlsafe</code> (~256 bits)
Enumeration	Infeasible at that entropy
Time-boxing	90 days by default, checked on read
Revocation	Immediate, by creating a replacement
Scope	One project, finalised transmittals only
Chrome	The template extends nothing; no navigation can leak in
Data exposed	Project number/title, transmittal references, drawing numbers/titles/revisions, issue dates

Risk note. The token appears in the URL and therefore in browser history and any intermediary logs. This is inherent to a link-based share model and was the brief's explicit requirement. Anyone holding the link has access until it expires or is replaced. There is no per-recipient tracking.

12.7 Security posture summary

CONTROL	STATUS
Password hashing	✓ Implemented
PIN hashing	✓ Implemented
CSRF protection	✓ Implemented
Role-based access control (server-side)	✓ Implemented
Object scoping	✓ Implemented
Auto-escaped templates	✓ Implemented
Audit trail	✓ Implemented
Soft delete	✓ Implemented
HTTPS enforcement	⚠ Deployment responsibility — not configured
Secure cookie flags	⚠ Not configured
PIN rate limiting	✗ Not implemented
Login rate limiting	✗ Not implemented
Two-factor authentication	✗ Not implemented (PIN is not 2FA for login)
File upload scanning	N/A — no uploads

13. Configuration

13.1 Settings layout

```
dscims/settings/
├─ base.py          shared by every environment
├─ dev.py           DEBUG=True, insecure key, console email
├─ production.py    DEBUG=False
└─ local.py         untracked, machine-specific (gitignored)
```

`manage.py`, `wsgi.py` and `asgi.py` default to `dscims.settings.dev`. Both `dev.py` and `production.py` end with an optional import of `local.py`.

Important. Configuration is **Python-based, not environment-variable based**. There is no `django-environ` or `.env` loading. Production values are supplied through `local.py` or by setting them in `production.py`. See §19.

13.2 ProjectIMS-specific settings

Branding is configuration. No template hard-codes a product or practice name. The three `DSCIMS_BRAND_*` settings drive the sidebar, page titles, the login screen, the transmittal PDF letterhead and the report print header. PDF templates are rendered without a request, so `core.pdf.render_pdf()` injects the brand explicitly rather than relying on the context processor.

SETTING	DEFAULT	PURPOSE
<code>DSCIMS_BRAND_NAME</code>	"ProjectIMS"	Product name shown in the sidebar, page titles, login and every generated document
<code>DSCIMS_BRAND_MARK</code>	"PI"	The two-letter square mark
<code>DSCIMS_BRAND_TAGLINE</code>	see settings	Subtitle on the login screen
<code>DSCIMS_PDF_ENGINE</code>	"auto"	<code>auto</code> / <code>weasyprint</code> / <code>playwright</code> / <code>html</code>
<code>DSCIMS_TRANSMITTAL_LOCK_MINUTES</code>	5	Edit-lock lifetime
<code>DSCIMS_SHARE_LINK_DAYS</code>	90	Client share-link lifetime
<code>DSCIMS_NOTIFICATIONS_ENABLED</code>	True	Default only — runtime value lives in <code>EmailSettings</code>
<code>DSCIMS_NOTIFICATIONS_FROM_NAME</code>	"ProjectIMS"	Default only
<code>DSCIMS_NOTIFICATIONS_FROM_EMAIL</code>	<code>dscims@example.com</code>	Default only
<code>DSCIMS_NOTIFICATIONS_REROUTE_TO</code>	""	Default only

The four notification settings are **initial defaults**. The live values are the `EmailSettings` singleton row, edited at `/admin/settings/`.

13.3 Framework settings of note

SETTING	VALUE	REASON
AUTH_USER_MODEL	accounts.Staff	Set before the first migration
SESSION_COOKIE_AGE	31536000 (1 year)	Brief's auto-login requirement
SESSION_SAVE_EVERY_REQUEST	True	Keeps the year rolling
TIME_ZONE	Australia/Sydney	⚠ Confirm with the client
LANGUAGE_CODE	en-au	
TEST_RUNNER	core.test_runner.DSCIMSTestRunner	Pins the PDF engine to <code>html</code> in tests

13.4 Production configuration template

Create `dscims/settings/local.py` on the server (never committed):

```
# dscims/settings/local.py – production values, NOT committed
SECRET_KEY = "<generate-a-new-50-character-random-key>"
DEBUG = False
ALLOWED_HOSTS = ["dscims.example.com"]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "<database-name>",
        "USER": "<database-user>",
        "PASSWORD": "<your-password>",
        "HOST": "<database-host>",
        "PORT": "5432",
        "CONN_MAX_AGE": 60,
    }
}

EMAIL_BACKEND = "django.core.mail.backends.smtp.EmailBackend"
EMAIL_HOST = "<smtp-host>"
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = "<smtp-username>"
EMAIL_HOST_PASSWORD = "<your-password>"

DSCIMS_PDF_ENGINE = "playwright" # pin explicitly rather than relying on auto

# Security hardening – required behind HTTPS
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
X_FRAME_OPTIONS = "DENY"
```

13.5 Application configuration performed in the UI

SETTING	WHERE	WHO
Email on/off, From name/address, reroute	/admin/settings/	IMS Coordinator
QA template content	/admin/templates/	IMS Coordinator
Customer Focus questions	/admin/questions/	IMS Coordinator
Staff and roles	/admin/staff/	IMS Coordinator
Project types, engagement types	Not exposed in the UI — see §19	—

14. Deployment Architecture

14.1 Environments

ENVIRONMENT	SETTINGS MODULE	DATABASE	DEBUG	PDF ENGINE
Development	dscims.settings.dev	SQLite	On	auto (Playwright)
Staging	dscims.settings.production + local.py	PostgreSQL	Off	Pin explicitly
Production	dscims.settings.production + local.py	PostgreSQL	Off	Pin explicitly

14.2 Target production topology



14.3 Deployment procedure

```

# 1. Code and dependencies
git pull
pip install -r requirements.txt
playwright install chromium          # only if using the Playwright engine

# 2. Configuration
#   Place dscims/settings/local.py with production values (never committed)

# 3. Database
python manage.py migrate --settings=dscims.settings.production

# 4. Static files
python manage.py collectstatic --noinput --settings=dscims.settings.production

# 5. Verification
python manage.py check --deploy --settings=dscims.settings.production

# 6. Restart the application server

```

14.4 First-run checklist

1. Create the first superuser (`createsuperuser`).
2. Sign in and set that account's category to **IMS Coordinator**.
3. Create staff records; each Manager sets a sign-off PIN via their profile.
4. Populate the **QA Template Library** — no project can be initialized without it.
5. Populate the **Question Master** — no survey can be created without it.
6. Create Project Types and Engagement Types (see §19 — currently requires the Django admin or a shell).
7. Configure email at `/admin/settings/` ; leave the reroute field **empty**.
8. Confirm `TIME_ZONE` .

14.5 Sizing and operational notes

- **Chromium is memory-hungry.** Each PDF render launches a browser process. On a small instance, concurrent exports can exhaust memory. Consider pinning `DSCIMS_PDF_ENGINE = "weasyprint"` on a Linux host, where GTK is readily available, for a substantially lighter footprint.
- **PDF rendering is synchronous.** A large report ties up a worker for its duration. Size the worker pool accordingly.
- **Email is synchronous.** A slow SMTP server slows the request that triggered it. `notify()` never raises, so a failure degrades to "logged, not sent".

14.6 Backup

The database is the entire system of record — there are no uploaded files to back up. Standard PostgreSQL backup practice applies. Audit tables (`NotificationLog` , `SignoffAttempt` , `PhaseSignoff` , `DrawingRevision`) are append-only and must be included.

15. Error Handling & Edge Cases

15.1 Validation errors

SCENARIO	RESPONSE
Mark a milestone complete with no date	Form error; not saved
Resolve an NC with no date or no resolver	Form error; NC stays open
Record an outcome with no completion date	Rejected; message shown
Record an outcome before a Pursue decision	Rejected even if the form is bypassed
Reissue a drawing at the same or a lower revision	"Revision must increase past X"; not added
Add a contact belonging to another company	Field error
Initialize with no disciplines	Field error

15.2 Permission and authentication errors

SCENARIO	RESPONSE
Unauthenticated request to a protected route	302 to <code>/login/?next=...</code>
Staff opening an admin screen	403 with an explanatory message
Staff attempting sign-off	403
Manager opening the template library	403
GET on a POST-only endpoint	405
Acting as a higher role	No effect — permissions follow the signed-in user

15.3 Missing or absent data

SCENARIO	BEHAVIOUR
Project with no checklist	Overview shows an empty state and an Initialize call to action
Report filters match nothing	Empty state, not an error
Portfolio with no data	Aggregations return 0 — no division by zero
Survey with no answers	Score 0 ; target not met
Question bank empty	Survey creation is blocked with an explanation
Template library empty	The wizard explains there is nothing to initialize from
No distribution list	The transmittal editor links to build one
No Manager has a PIN	The sign-off screen explains and hides the confirm button
Project/Engagement types not created	Fields are optional and simply omitted

15.4 Invalid state and concurrency

SCENARIO	BEHAVIOUR
Two users edit one transmittal	Second user is read-only until the lock expires
Editing a signed-off phase	Activity rows show "Locked"; no change is made
Finalising an already-finalised transmittal	<code>TransmittalError</code> ; message shown
Signing an already-signed phase	Rejected; attempt logged
Revision overtaken between adding and finalising	Caught again at finalise
Acting user deactivated mid-session	Silent fallback to the signed-in account
Editing a resolved NC	Confirmation prompt before saving

15.5 External service failures

FAILURE	BEHAVIOUR
SMTP unavailable	Notification logged with <code>was_emailed = False</code> ; user's action still succeeds
PDF engine missing	Falls back to HTML; the screen says so
Playwright installed without Chromium	Caught; falls through to the next engine
Google Fonts / cdnjs unreachable	Fonts degrade to system stack; HTMX and Alpine fail — see §19

15.6 Unhandled errors

With `DEBUG = False` , Django serves standard 404 and 500 pages. **No custom error templates and no error-monitoring integration (e.g. Sentry) are configured.** See §19.

16. End-to-End Business Scenarios

Scenario A — Commissioning ProjectIMS for the first time

#	ACTOR	ACTION	SYSTEM PROCESSING	RESULT
1	DevOps	Deploy, migrate, collectstatic	Reference data migration seeds 7 disciplines and 5 QA types	Application live
2	DevOps	createsuperuser	Staff row created; ST-001 allocated	First account exists
3	IMS Coordinator	Set own category to IMS Coordinator	—	Admin section appears
4	IMS Coordinator	Create staff records	Each gets a sequential ST-nnn	Team exists
5	Manager	Set a sign-off PIN in Profile	PIN hashed	Eligible to sign off
6	IMS Coordinator	Build the template library	Phases/tasks/activities created	Projects can be initialized
7	IMS Coordinator	Add Customer Focus questions	—	Surveys can be created
8	IMS Coordinator	Configure email	EmailSettings row written	Notifications dispatch

Scenario B — A project from intake to first sign-off

#	ACTOR	ACTION	SYSTEM PROCESSING	RESULT
1	PM	Create project 2118, tick H/M/E/F with engineers	<code>Project</code> + 4 <code>ProjectService</code> rows; <code>created_by</code> = acting user	Project exists, "Not initialized"
2	PM	Wizard step 1 — choose disciplines	Stored in session	Step 2
3	PM	Step 2 — choose phases	Union of discipline + Common phases offered	Step 3
4	PM	Step 3 — confirm (amber: non-destructive)	<code>initialize_project()</code> builds the tree in a transaction	20 phases, 48 activities
5	Engineer	Open the Hydraulic checklist	—	Tree displayed
6	Engineer	Mark activities Done	HTMX; row swaps, header refreshes out-of-band	Progress rises live
7	System	All activities in a task settled	Derived	Task shows "Closed"
8	System	All tasks in the phase closed	Derived	"Ready to sign off"
9	Manager	Open sign-off, enter a wrong PIN	Failed <code>SignoffAttempt</code> written	"That PIN is not correct."
10	Manager	Enter the correct PIN	Successful attempt, <code>PhaseSignoff</code> , reconciliation	Phase signed; activities locked
11	System	Phase is Common	Mirrors onto M/E/F with <code>reconciled_from</code>	Signed across four disciplines
12	System	—	<code>notify(phase_signed_off)</code>	Appears in the feed as "Recorded only"

Scenario C — A non-conformance from discovery to closure

#	ACTOR	ACTION	SYSTEM PROCESSING	RESULT
1	Engineer	Raise an NC	<code>NC-001</code> allocated from the global counter	NC created, status Open
2	System	—	<code>notify(nc_raised)</code> to all staff	"Every staff member has been notified"
3	Engineer	Record investigation and corrective action	Saved; no new notification	Narrative populated
4	Engineer	Resolve with no date	Form rejects	NC stays open
5	Engineer	Resolve with resolver + date	<code>resolved_by</code> , <code>resolved_date</code> set	Status → Resolved
6	System	—	<code>notify(nc_resolved)</code> to all staff	Recorded
7	Manager	NC report	Appears with dates and status	Available for export

Scenario D — Issuing drawings to a client

#	ACTOR	ACTION	SYSTEM PROCESSING	RESULT
1	Engineer	Add companies and contacts	—	Address book populated
2	Engineer	Build the distribution list	<code>ProjectCompany</code> rows with delivery methods	List ready
3	Engineer	New transmittal	<code>TR2118-001</code> allocated (per project); lock taken	Draft open
4	Engineer	Tag H-2118-101 at rev C	<code>previous_revision</code> captured; increase validated	Green "C ✓"
5	Colleague	Opens the same transmittal	Lock held	Read-only, "locked to J. Smith for 4 more minutes"
6	Engineer	Add three recipients	Copied from the distribution list	Recipients listed
7	Engineer	Link MS-001	—	Milestone linked
8	Engineer	Finalise & issue	Verify → snapshot → <code>DrawingRevision</code> → stamp → PDF → notify	Immutable
9	System	All linked transmittals finalised	<code>check_milestone_completion()</code>	MS-001 auto-completes
10	System	—	Two notifications: external issue, internal finalised	Recorded
11	Engineer	Create a share link	32-byte token; previous link revoked	Link valid 90 days
12	Client	Opens the link, no login	Token validated, expiry checked	Sees the finalised transmittal only
13	Engineer	Attempts to reissue rev C	Rejected	"Revision must increase past C"

Scenario E — Correcting a template error already in live projects

#	ACTOR	ACTION	SYSTEM PROCESSING	RESULT
1	IMS Coordinator	Notices wrong wording on an activity	—	—
2	IMS Coordinator	Edits the template activity	Saved; live projects unchanged	Future initializations correct
3	IMS Coordinator	Opens Re-sync, selects projects	<code>preview()</code> — reads only	Diff of 4 changes shown
4	IMS Coordinator	Applies	<code>apply_resync()</code> writes wording and links	Live checklists corrected
5	System	—	Ticked states untouched	Progress preserved

Scenario F — A survey that reaches its target mid-entry

#	ACTOR	ACTION	SYSTEM PROCESSING	RESULT
1	PM	Create the survey	Target snapshotted; questions copied	CF-001 with 5 questions
2	PM	Score 9, 8, 7	Each saved individually; cache dropped	Running score 80%
3	System	80% ≥ 75% target	completed_at stamped	Auto-completed
4	System	—	notify(cf_completed)	Recorded
5	PM	Scores the remaining two	Saved	Final 82%; stays complete
6	IMS Coordinator	Retires a bank question	is_active = False	Existing survey unchanged

17. Maintenance & Future Development Guide

17.1 Orientation for a new engineer

Read in this order:

1. `PROGRESS.md` — the development handoff, including "do not undo these" decisions.
2. `dscims_build_prompt.md` — the original client brief.
3. `design-reference/` — 17 approved screen designs; the source of truth for UI.
4. This document.

Then run the test suite. **342 tests in roughly 20 seconds** is the fastest way to confirm a working environment.

17.2 Where functionality lives

TO CHANGE...	LOOK IN...
A permission rule	The view's explicit check; <code>Staff.is_manager</code> / <code>is_administrator</code>
Who receives a notification	<code>notifications/services.py</code> → <code>AUDIENCES</code>
Checklist building	<code>qa/services.py</code>
Sign-off rules	<code>qa/signoff.py</code> → <code>_rejection_reason()</code>
Re-sync scope	<code>qa/resync.py</code>
Transmittal rules	<code>documents/models.py</code> → <code>can_finalize()</code> ; <code>documents/services.py</code>
Revision comparison	<code>documents/models.py</code> → <code>revision_sort_key()</code>
Report filters	<code>reports/forms.py</code>
Portfolio aggregation	<code>reports/queries.py</code>
PDF engine	<code>core/pdf.py</code> ; <code>DSCIMS_PDF_ENGINE</code>
Visual design	<code>dscims/static/dscims/css/dscims.css</code> (extracted) and <code>app.css</code> (additions)

17.3 How to add a feature

1. **Model** — add to the owning app; keep derived state as properties, not stored fields.
2. **Migration** — `makemigrations`, review the generated file, `migrate`.
3. **Form** — reuse `RequiresCompletionDateMixin` if it completes something.
4. **View** — add the explicit role check; use `@require_POST` for state changes.
5. **URL** — add to the app's `urls.py`.
6. **Template** — extend `base.html`; reuse `core/_field.html` and the design tokens.
7. **Notification** — add a trigger to `NotificationTrigger` **and** an entry to `AUDIENCES`; call `notify()`.
8. **Tests** — model behaviour, view role gating, and a failure path.

17.4 Adding a notification — worked example

```
# 1. notifications/models.py
class NotificationTrigger(models.TextChoices):
    ...
    DRAWING_SUPERSEDED = "drawing_superseded", "Drawing superseded"

# 2. notifications/services.py — an unmapped trigger silently becomes
# "Recorded only", so this step is easy to forget and must not be.
AUDIENCES = {
    ...
    Trigger.DRAWING_SUPERSEDED: ("Project team", "project"),
}

# 3. At the call site
notify(Trigger.DRAWING_SUPERSEDED,
      f"{drawing.number} superseded by rev {revision}.",
      project=project, actor=request.acting_user)
```

17.5 Areas requiring extra care

AREA	WHY	GUIDANCE
<code>qa/signoff.py</code>	Validation is outside the transaction on purpose	Moving it inside silently stops failed attempts being recorded
<code>core/Counter</code>	<code>select_for_update()</code> is a no-op on SQLite	Test concurrency against PostgreSQL
Common-phase reconciliation	Subtle, and bidirectional on revoke	Extend the existing tests before changing
<code>CustomerFocus.answered_scores</code>	<code>cached_property</code>	A request that writes an answer must drop the cache
<code>reports/queries.py</code>	Bulk aggregation	Replacing with per-row properties reintroduces N+1
Recipient snapshotting	Historical accuracy	Never "improve" it by reading live company data
<code>dscims.css</code>	Extracted verbatim from the design pack	Put additions in <code>app.css</code> , not here
Template comments	<code>{# ... #}</code> is single-line only	A multi-line one renders as visible text — this was made twice
Filter arguments	<code>{{ a.b\ default:c.d }}</code> raises when <code>c</code> is None	Use <code>{% firstof %}</code>

17.6 Testing conventions

Django's own test runner (not pytest), `tests/` packages with one file per concern, hand-written factory helpers in `accounts/tests/utils.py`, and `fast_password_hashing` for login-heavy tests.

The runner pins `DSCIMS_PDF_ENGINE = "html1"` so tests never launch a browser. A test needing a specific engine overrides the setting itself.

18. Troubleshooting

SYMPTOM	LIKELY CAUSE	RESOLUTION
Exports arrive as <code>.html</code>	No PDF engine available	<code>pip install playwright && playwright install chromium</code> , or pin an engine
<code>OSError: cannot load library 'libgobject-2.0-0'</code>	WeasyPrint installed without GTK	Install the GTK runtime, or use Playwright. Do not leave WeasyPrint installed but broken — failed imports are not cached, so every render retries and prints a warning
Test suite suddenly takes minutes	<code>TEST_RUNNER</code> overridden or removed	Restore <code>core.test_runner.DSCIMSTestRunner</code>
Nothing is emailed; feed says "Logged only"	Staff have no email addresses, or the master switch is off	Populate emails; check <code>/admin/settings/</code>
Client reports emails arriving at one address	Debug reroute is set	Clear <code>reroute_to</code> ; a red banner shows on Settings
Sign-off button never appears	Phase not eligible, or user is not a Manager	Close every task; check category
Signatory dropdown is empty	No Manager has a PIN	Managers set one in Profile
Wizard offers no phases	Template library empty for those disciplines	Populate the library
"Revision must increase past X"	The revision was already issued	Use the next revision
Transmittal read-only unexpectedly	Someone else holds the lock	Wait up to 5 minutes
Share link 404s	Expired, revoked, or replaced	Create a new link
Comment text visible on a page	Multi-line <code>{# ... #}</code>	Convert to <code>{% comment %}</code>
<code>VariableDoesNotExist</code> in a template	Failed lookup used as a filter argument	Use <code>{% firstof %}</code>
Sidebar missing expected sections	Category is wrong, or a legacy value is stored	Check <code>Staff.category</code>
Bash prints <code>virtualenvwrapper...</code> noise	Malformed <code>WORKON_HOME</code> in the Windows shell profile	Cosmetic; ignore or fix the profile

19. Known Limitations & Specification Gaps

19.1 Specification gaps requiring client confirmation

#	GAP	CURRENT IMPLEMENTATION	IMPACT
G-1	The PRD referenced by the brief was never supplied — its permissions matrix and notification matrix are the authoritative source	Roles derived from the brief's per-screen role column; audiences from per-module rules, each annotated in <code>AUDIENCES</code>	Medium — reconciling is a table edit, but the current audiences are inferences
G-2	Reconciled sign-off semantics	Signing a Common phase marks it signed in every discipline, per brief §5.3. Because each discipline holds its own copy of the Common activities, reconciled phases can be signed while their own activities are unticked; the UI labels these " (reconciled)"	Medium — the alternative (one shared activity set) is a data-model change
G-3	<code>NonConformance.caused_by</code>	Free text, per the approved screen ("Structural — column C4 relocation not communicated"). The brief calls it part of a "four-person attribution block"	Low — changing to a foreign key would lose expressiveness
G-4	Additional Scope notification audience	Project team. Brief §5.5 says "Eng/Director/Associate" — roles the missing matrix would define	Low
G-5	Project Types and Engagement Types	Deliberately not seeded — neither the brief nor the designs enumerate them	Low — but see §19.2

19.2 Functional limitations

#	LIMITATION	DETAIL
L-1	No UI for Project/Engagement Types	The taxonomies exist and are used, but must be populated via the Django admin or a shell. Fields remain optional, so intake is not blocked
L-2	Drawing files are not stored	Only metadata and revision history. <code>MEDIA_ROOT</code> is configured but unused
L-3	Transmittal PDFs are not persisted	Regenerated on each request. A re-render after a data change would not match the issued document byte-for-byte
L-4	Share links are project-wide	Not per transmittal, and there is no per-recipient tracking
L-5	Milestone "spawn new milestone inline"	The transmittal editor links to existing milestones and offers a link to create one; it does not create one inline as the artboard suggests
L-6	No template drag-to-reorder	Ordering is by an integer field, edited numerically
L-7	Notification feed is global	Not filtered per user; capped at 200 rows
L-8	No data retention or archival	Audit tables grow without bound

19.3 Technical debt and operational risk

#	ITEM	RISK	RECOMMENDATION
T-1	Configuration is Python files, not environment variables	Medium — poor fit for containers and 12-factor deployment	Introduce <code>django-environ</code> or <code>os.environ</code> reads in <code>production.py</code>
T-2	SQLite in development, PostgreSQL in production	Medium — <code>select_for_update()</code> is a no-op on SQLite, so counter concurrency is untested locally	Run the suite against PostgreSQL in CI
T-3	Monotonic revision rule is not a database constraint	Low-medium — a direct database write could violate it	Accept, or add a trigger
T-4	No rate limiting on login or PIN attempts	Medium-high	Add <code>django-ratelimit</code> or equivalent before external exposure
T-5	Security cookie flags not set	High if deployed as-is	Apply the §13.4 hardening block
T-6	Synchronous email and PDF generation	Medium — slow SMTP or large reports tie up a worker	Introduce a task queue if volume grows
T-7	HTMX and Alpine load from a public CDN	Medium — the app degrades badly if cdnjs is blocked	Vendor them into static files
T-8	No error monitoring	Medium — production failures are invisible	Add Sentry or equivalent
T-9	No custom 404/500 templates	Low	Add branded error pages
T-10	Django admin exposed with no registered models	Low	Remove the route in production if unused
T-11	Chromium memory footprint	Medium	Pin WeasyPrint on Linux, or size instances accordingly
T-12	<code>TIME_ZONE</code> assumed Australia/Sydney	Low	Confirm with the client

19.4 What is explicitly not implemented

- No REST or GraphQL API
- No background workers, task queue or scheduled jobs
- No file upload or object storage
- No two-factor authentication for login
- No client-facing portal beyond the read-only share link
- No legacy data migration from the Yii application
- No multi-tenancy — one practice per deployment
- No internationalisation beyond `en-au`
- No soft delete for projects or register entries (status only)

20. Glossary

TERM	MEANING
Acting user	The identity work is attributed to. Changed by the "Working with..." switcher. Never affects permissions
Activity	The leaf of the QA checklist; the thing an engineer ticks (<code>SubTaskMaster</code> / <code>ProjectGroupSubTask</code>)
Additional Scope (AS)	Work beyond the base contract; lifecycle Raised → Decision → Outcome
Background	An external consultant's drawing that the practice coordinates against; tracked in the Document Register
Common discipline	A pseudo-discipline whose phases are folded into every other discipline at initialization
Common phase	A phase originating from the Common discipline; signing one reconciles across all disciplines
Customer Focus (CF)	The client satisfaction survey; scored 0–10 per question
Discipline	An engineering specialism: Hydraulic, Mechanical, Electrical, Fire, Lift, Sustainability, Common
Distribution list	The per-project subset of the address book that transmittal recipients are drawn from
ProjectIMS	ProjectIMS — QA and document control
Edit lock	A short, self-expiring hold on a transmittal draft
Finalise	The irreversible act of issuing a transmittal
IMS Coordinator	The administrator role; <code>is_administrator</code> in code
Initialization	Building a project's live checklist from the template library. Destructive when re-run
Invalid	An activity state meaning superseded. Does not count as complete
Live checklist	A project's own copy of the QA tree, created by initialization
Milestone (MS)	A dated issue commitment; may auto-complete via transmittals
Non-Conformance (NC)	A recorded QA failure. Raising one notifies every staff member
N/A	An activity state meaning not applicable. Does count as settled
Phase	The top level of the checklist (<code>GroupMaster</code> / <code>ProjectGroup</code>), e.g. Initialisation, Review
PIN	The short hashed code confirming a phase sign-off
Question Master	The firm-wide Customer Focus question bank
Reconciled	A sign-off mirrored onto another discipline's copy of a Common phase
Re-sync	Pushing template wording and link corrections into live projects
Settled	An activity that is Done or N/A; the numerator of every progress figure
Share link	A time-boxed unauthenticated URL exposing a project's finalised transmittals
Snapshot	Copying a value at a moment so later edits cannot rewrite history
Task	The middle level of the checklist; auto-closes when all its activities are settled
Template library	The admin-owned QA tree from which live checklists are copied

TERM	MEANING
Transmittal (TR)	An issue event: drawings at revisions, to recipients, on a date

21. Appendix A — Complete Route Table

69 application routes. All require an authenticated session except where noted.

Authentication & profile

METHOD	URL	NAME	ROLE
GET/POST	/login/	accounts:login	Public
GET	/logout/	accounts:logout	Public
POST	/switch-identity/	accounts:switch_identity	Any
GET/POST	/profile/	accounts:profile	Any

Dashboard, settings & notifications

METHOD	URL	NAME	ROLE
GET	/	core:dashboard	Any
GET/POST	/admin/settings/	core:settings	IMS Coordinator
GET	/notifications/	notifications:feed	Any

Projects

METHOD	URL	NAME	ROLE
GET	/projects/	projects:list	Any
GET/POST	/projects/new/	projects:create	Any
GET	/projects/<pk>/	projects:detail	Any
GET/POST	/projects/<pk>/initialize/	projects:initialize	Any
POST	/projects/<pk>/favorite/	projects:toggle_favorite	Any

QA checklist & sign-off

METHOD	URL	NAME	ROLE
GET	/projects/<pk>/checklist/<code>/	qa:checklist	Any
POST	/checklist/activity/<pk>/state/	qa:set_activity_state	Any
GET/POST	/checklist/phase/<pk>/signoff/	qa:phase_signoff	Manager
POST	/checklist/phase/<pk>/unsign/	qa:phase_unsign	Manager

QA template library (all IMS Coordinator)

METHOD	URL	NAME
GET	/admin/templates/	qa:template_library
GET	/admin/templates/d/<code>/	qa:template_library_discipline
GET/POST	/admin/templates/resync/	qa:template_resync
GET/POST	/admin/templates/phase/new/	qa:phase_create
GET/POST	/admin/templates/phase/<pk>/	qa:phase_edit
GET/POST	/admin/templates/phase/<pk>/task/new/	qa:task_create
GET/POST	/admin/templates/task/<pk>/	qa:task_edit
GET/POST	/admin/templates/task/<pk>/activity/new/	qa:activity_create
GET/POST	/admin/templates/activity/<pk>/	qa:activity_edit
POST	/admin/templates/<kind>/<pk>/delete/	qa:template_delete

Registers

METHOD	URL	NAME	ROLE
GET	/projects/<pk>/milestones/	registers:milestone_list	Any
GET/POST	/projects/<pk>/milestones/new/	registers:milestone_create	Any
GET/POST	/projects/<pk>/milestones/<pk>/	registers:milestone_detail	Any
GET	/projects/<pk>/non-conformances/	registers:nc_list	Any
GET/POST	/projects/<pk>/non-conformances/new/	registers:nc_create	Any
GET/POST	/projects/<pk>/non-conformances/<pk>/	registers:nc_detail	Any
POST	/projects/<pk>/non-conformances/<pk>/resolve/	registers:nc_resolve	Any
GET	/projects/<pk>/additional-scope/	registers:as_list	Any
GET/POST	/projects/<pk>/additional-scope/new/	registers:as_create	Any
GET/POST	/projects/<pk>/additional-scope/<pk>/	registers:as_detail	Any
POST	/projects/<pk>/additional-scope/<pk>/decide/	registers:as_decide	Any
POST	/projects/<pk>/additional-scope/<pk>/outcome/	registers:as_outcome	Any
GET/POST	/projects/<pk>/customer-focus/	registers:cf_detail	Any
POST	/projects/<pk>/customer-focus/answer/<pk>/	registers:cf_answer	Any
GET/POST	/admin/questions/	registers:question_master	IMS Coordinator
POST	/admin/questions/<pk>/toggle/	registers:question_toggle	IMS Coordinator

Document control

METHOD	URL	NAME	ROLE
GET	/projects/<pk>/transmittals/	documents:transmittal_list	Any
GET	/projects/<pk>/transmittals/new/	documents:transmittal_create	Any
GET/POST	/projects/<pk>/transmittals/<pk>/	documents:transmittal_detail	Any
POST	/projects/<pk>/transmittals/<pk>/finalise/	documents:transmittal_finalize	Any
GET	/projects/<pk>/transmittals/<pk>/pdf/	documents:transmittal_pdf	Any
GET/POST	/projects/<pk>/drawings/new/	documents:drawing_create	Any
GET/POST	/projects/<pk>/document-register/	documents:document_register	Any
GET/POST	/projects/<pk>/document-register/<pk>/	documents:background_edit	Any
GET/POST	/projects/<pk>/companies/	documents:distribution_list	Any
POST	/projects/<pk>/companies/<pk>/remove/	documents:distribution_remove	Any
POST	/projects/<pk>/share-link/	documents:share_link_create	Any
GET	/share/<token>/	documents:shared_project	Public

Companies

METHOD	URL	NAME	ROLE
GET	/companies/	companies:list	Any
GET/POST	/companies/new/	companies:create	Any
GET	/companies/<pk>/	companies:detail	Any
GET/POST	/companies/<pk>/edit/	companies:edit	Any
GET/POST	/companies/<pk>/contacts/new/	companies:contact_create	Any
GET/POST	/companies/contacts/<pk>/	companies:contact_edit	Any

Reports (all Manager or IMS Coordinator)

METHOD	URL	NAME
GET	/reports/	reports:hub
GET	/reports/non-conformances/	reports:nc
GET	/reports/additional-scope/	reports:additional_scope
GET	/reports/customer-feedback/	reports:customer_feedback
GET	/reports/activity-status/	reports:activity_status
GET	/reports/summaries/	reports:summaries

All report routes accept `?export=pdf` and the shared filter parameters.

22. Appendix B — Model Index

APP	MODEL	PURPOSE
core	Counter	Gap-free named sequences
core	EmailSettings	Runtime email configuration (singleton)
accounts	Staff	The only user model
projects	Discipline	Engineering discipline taxonomy (7 seeded)
projects	QAType	QA classification (5 seeded)
projects	ProjectType	Delivery model — not seeded
projects	EngagementType	Engagement model — not seeded
projects	Project	The hub record
projects	ProjectService	Discipline + assigned engineer
projects	Favorite	Per-staff starred projects
qa	GroupMaster	Template phase
qa	TaskMaster	Template task
qa	SubTaskMaster	Template activity, with applicability and references
qa	ProjectGroup	Live phase
qa	ProjectGroupTask	Live task
qa	ProjectGroupSubTask	Live activity, carrying state
qa	PhaseSignoff	A signature, with revocation and reconciliation links
qa	SignoffAttempt	Every PIN attempt
registers	Milestone	Dated issue commitment
registers	NonConformance	Recorded QA failure
registers	AdditionalScope	Out-of-contract work
registers	QuestionMaster	Firm-wide survey question bank
registers	CustomerFocus	Per-project survey
registers	CustomerFocusAnswer	One answer, with snapshotted wording
companies	Company	External organisation
companies	Contact	Person at a company
companies	ProjectCompany	Distribution list entry
documents	Drawing	The practice's own drawing
documents	DrawingRevision	An issued revision
documents	TransmittalEvent	An issue event

APP	MODEL	PURPOSE
documents	TransmittalItem	A drawing at a revision on a transmittal
documents	TransmittalRecipient	A snapshotted recipient
documents	BackgroundDocument	External consultant drawing
documents	ShareLink	Time-boxed client access
notifications	NotificationLog	One row per notification fired

23. Appendix C — Test Inventory

342 tests, executing in approximately 20 seconds.

AREA	COVERAGE
core	Counter sequencing, reference allocation, PDF engine selection and fallback
accounts	Staff numbering, display names, PIN hashing, roles, login, identity switching, staff admin, profile
projects	Status derivation, search and filters, favourites, detail rendering
qa	Initialization, Common folding, applicability, ordering, re-sync, task auto-close, progress, sign-off, reconciliation, revocation, role gating
registers	References, all four status derivations, CF scoring and snapshotting, completion-date guard, lifecycle gating, notifications
companies	Covered through the document-control tests
documents	Revision ordering, edit locks, share-link validity, finalise guards and effects, recipient snapshotting, milestone auto-completion, forced notifications
notifications	Audience resolution, suppression, rerouting, send-failure resilience, feed
reports	Filtering, aggregation, team workload, role gating, export serving

Run with:

```
python manage.py test
```

End of document.

ProjectIMS — Project System Documentation & Engineering Handover, version 1.0. Prepared from direct inspection of the implemented source. Where behaviour is configuration-dependent, inferred from an incomplete specification, or not implemented, this document says so.